

Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης  
Πολυτεχνική Σχολή  
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών  
Τομέας Ηλεκτρονικής και Υπολογιστών

---



ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ  
Διαμέριση γράφων

---

Θεόδωρος Κεσόγλου

Επιβλέπων: Νικόλαος Ατρέας  
Αναπληρωτής Καθηγητής  
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών  
Α.Π.Θ.

Θεσσαλονίκη, Ελλάδα, 2025

Aristotle University of Thessaloniki  
Faculty of Engineering  
Department of Electrical and Computer Engineering



---

DIPLOMA THESIS  
Graph Partitioning

---

Theodoros Kesoglou

Supervisor: Associate Prof. Nikolaos Atreas  
School of Electrical and Computer Engineering  
Aristotle University of Thessaloniki

Thessaloniki, Greece, 2025

# Περίληψη

Στην παρούσα διπλωματική εργασία ασχολούμαστε με αλγορίθμους διαμέρισης γράφων. Η διαμέριση γράφου είναι ένα θεμελιώδες πρόβλημα της συνδυαστικής βελτιστοποίησης και έχει στόχο την κατάτμηση ενός γράφου σε υποσύνολα κόμβων, σύμφωνα με ορισμένα κριτήρια. Συγκεκριμένα, ένας γράφος με βάρη στις ακμές, ζητείται να χωριστεί σε  $k$  υποσύνολα ορισμένου μεγέθους, έτσι ώστε να ελαχιστοποιείται το συνολικό βάρος των ακμών που συνδέουν κόμβους διαφορετικών ομάδων.

Στο κεφάλαιο 1 παραθέτουμε στοιχεία ορολογίας της θεωρίας γράφων. Ξεκινάμε με τον ορισμό του γράφου, συνδέουμε τον γράφο με τον πίνακα γειτνίασης και συνεχίζουμε με τον ορισμό της Λαπλασιανής. Επίσης, παραθέτουμε και κάποια εργαλεία της γραμμικής άλγεβρας για την κωδικοποίηση γράφων.

Στο κεφάλαιο 2 μελετούμε δύο βασικούς αλγορίθμους φασματικής συσταδοποίησης γράφων: τον αλγόριθμο φασματικής συσταδοποίησης Spectral Clustering και τον αλγόριθμο κανονικοποιημένης τομής Normalized Cut. Και οι δύο μέθοδοι χρησιμοποιούν τη φασματική ανάλυση του γραφήματος, δηλαδή την ανάλυση ιδιοτιμών και ιδιοδιανυσμάτων της Λαπλασιανής, προκειμένου να διαμερίσουν (σε μη κενά ξένα μεταξύ τους υποσύνολα κόμβων) το γράφο.

Στο κεφάλαιο 3 αναλύουμε τον αλγόριθμο των Kernighan-Lin και τη δική μας παραλλαγή του αλγορίθμου. Έπειτα κάνουμε μια σύγκριση ανάμεσα σε αυτούς του αλγορίθμους και τον αλγόριθμο Normalize-Cut.

**Λέξεις-Κλειδιά:** Διαμέριση γράφου, Λαπλασιανή, Φασματική συσταδοποίηση, Κανονικοποιημένη τομή, αλγόριθμος Kernighan-Lin

# Abstract

In this thesis, we study graph partitioning algorithms. Graph partitioning is a fundamental problem in combinatorial optimization, aiming to divide a graph into subsets of nodes according to specific criteria. More precisely, given a graph with weighted edges, the objective is to divide it into  $k$  subsets of pre-determined size in such a way that the total weight of the edges connecting nodes from different groups is minimized.

Chapter 1 provides introductory terminology from graph theory. We begin with the definition of a graph, proceed to its representation via the adjacency matrix, and continue with the definition of the graph Laplacian. We also present some tools from linear algebra used in the encoding of graphs.

Chapter 2, we study two fundamental spectral clustering algorithms for graphs: the Spectral Clustering algorithm and the Normalized Cut algorithm. Both methods rely on the spectral analysis of the graph, i.e., the eigenvalue and eigenvector decomposition of the Laplacian matrix, to partition the graph into non-empty, disjoint subsets of nodes.

Chapter 3, we analyze the Kernighan–Lin algorithm and our own variation of it. Finally, we compare these algorithms with the Normalized Cut algorithm.

**Keywords:** Graph partitioning, Laplacian, Spectral Clustering, Normalized Cut, Kernighan-Lin algorithm.

## Ευχαριστίες

Θα ήθελα να ευχαριστήσω την οικογένεια μου για την στήριξη τους όλα αυτά τα χρόνια καθώς και τους κοντινούς μου φίλους.

Επίσης θα ήθελα να ευχαριστήσω θερμά τον καθηγητή και επιβλέποντα μου, κ. Νικόλαο Ατρέα για την καθοδήγηση και την υπομονή του καθ'όλη τη διάρκεια της διπλωματικής.

# Περιεχόμενα

|                                                       |           |
|-------------------------------------------------------|-----------|
| Περίληψη                                              | i         |
| Abstract                                              | ii        |
| Ευχαριστίες                                           | iii       |
| Περιεχόμενα                                           | iv        |
| <b>1 Εισαγωγή</b>                                     | <b>1</b>  |
| 1.1 Στοιχεία Θεωρίας Γράφων . . . . .                 | 1         |
| 1.2 Φασματική Ανάλυση του Πίνακα Γειτνίασης . . . . . | 7         |
| <b>2 Φασματική συσταδοποίηση γράφων</b>               | <b>11</b> |
| 2.1 Αλγόριθμος Spectral Clustering . . . . .          | 11        |
| 2.2 Αλγόριθμοι Graph Cut . . . . .                    | 18        |
| <b>3 Αλγόριθμος Kernighan–Lin</b>                     | <b>29</b> |
| 3.1 Εισαγωγή . . . . .                                | 29        |
| 3.2 Μια παραλλαγή του αλγορίθμου . . . . .            | 35        |
| 3.3 Σύγκριση μεταξύ των αλγορίθμων . . . . .          | 42        |

# 1 Εισαγωγή

Σε αυτό το κεφάλαιο παραθέτουμε στοιχεία ορολογίας της θεωρίας γράφων. Ξεκινάμε με τον ορισμό του γράφου, συνδέουμε τον γράφο με τον πίνακα γειτνίασης και συνεχίζουμε με την Λαπλασιανή. Επίσης, παραθέτουμε και κάποια εργαλεία της γραμμικής άλγεβρας για την κωδικοποίηση γράφων.

Η θεωρία γράφων αποτελεί μια από τις βασικές έννοιες της σύγχρονης υπολογιστικής επιστήμης και των εφαρμοσμένων μαθηματικών. Οι γράφοι χρησιμοποιούνται για να περιγράψουν πολύπλοκες σχέσεις και συνδέσεις μεταξύ στοιχείων/αντικειμένων σε πεδία όπως τα δίκτυα υπολογιστών, τα κοινωνικά δίκτυα, η βιολογία, η επιστήμη των δεδομένων και η θεωρία πληροφορίας. Η δομή ενός γράφου αποτυπώνεται από τον πίνακα γειτνίασης. Μέσω της φασματικής ανάλυσης δηλαδή της αποσύνθεσης των ιδιοτιμών και των ιδιοδιανυσμάτων του πίνακα γειτνίασης και της Λαπλασιανής, μπορούμε να μελετήσουμε χαρακτηριστικά και ιδιότητες ενός γράφου.

## 1.1 Στοιχεία Θεωρίας Γράφων

Στην παράγραφο αυτή χρησιμοποιούμε τα συγγράμματα [1, 2, 3].

**Ορισμός 1.1.** Καλούμε **γράφο**  $G$  να είναι το ζεύγος  $G = (V, E)$ , όπου  $V$  είναι ένα πεπερασμένο σύνολο  $n$  στοιχείων και  $E$  ένα πεπερασμένο σύνολο  $m$  διατεταγμένων ζευγών με στοιχεία του συνόλου  $V$ .

Τα στοιχεία του συνόλου  $V$  ονομάζονται κορυφές ενώ τα στοιχεία του συνόλου  $E$  ακμές. Κάθε ακμή είναι ένα διατεταγμένο ζεύγος  $(u, v)$  που συνδέει τις κορυφές  $u$  και  $v$ .

Λέμε ότι οι γράφοι είναι:

- **Μη κατευθυνόμενοι**, εάν οι ακμές του γράφου δεν έχουν κατεύθυνση.
- **Κατευθυνόμενοι**, εάν υπάρχει κατεύθυνση από την μια κορυφή προς μια άλλη (π.χ. από την  $u$  στην  $v$ ,  $u \rightarrow v$ ).
- **Με βάρη**, όταν οι ακμές φέρουν στάθμιση την οποία καλούμε βάρος.
- **Συνεκτικοί**, εάν υπάρχει μονοπάτι μεταξύ κάθε ζεύγους κορυφών.
- **Κανονικοί**, όταν όλες οι κορυφές συνδέονται με τον ίδιο αριθμό ακμών, τον οποίο κοινό αυτό αριθμό καλούμε βαθμό.

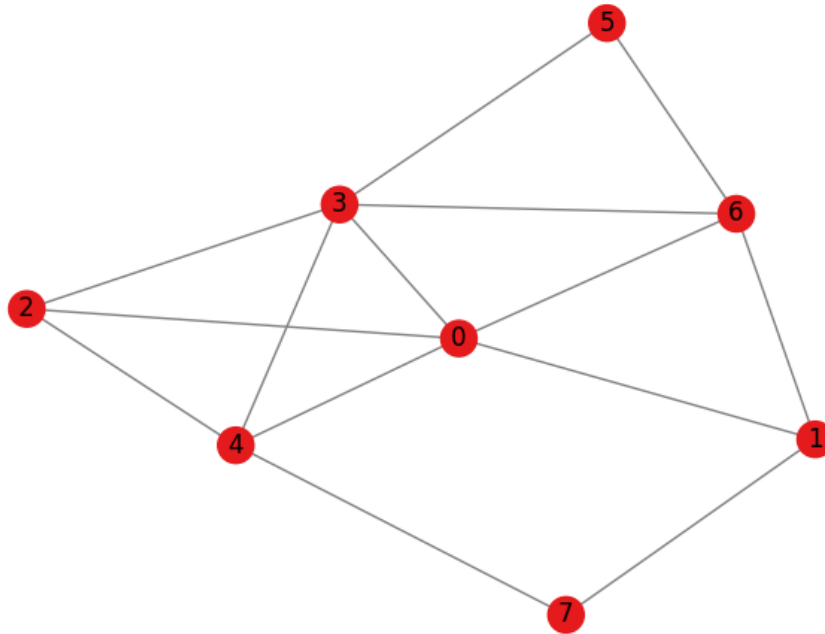
**Ορισμός 1.2.** Ο **Πίνακας Γειτνίασης**  $A$  ενός γράφου  $G = (V, E)$  με  $N$  κορυφές είναι ένας  $N \times N$  πίνακας, τέτοιος ώστε :

$$A_{ij} = \begin{cases} 1, & (i, j) \in E \\ 0, & (i, j) \notin E \end{cases}.$$

Ο πίνακας  $A$  περιγράφει την συνδεσιμότητα των κορυφών.

Αν η τιμή  $A_{ij} = 0$ , τότε οι κορυφές  $i$  και  $j$  δεν ενώνονται με κάποια ακμή, ενώ αν  $A_{ij} = 1$ , οι κορυφές επικοινωνούν.

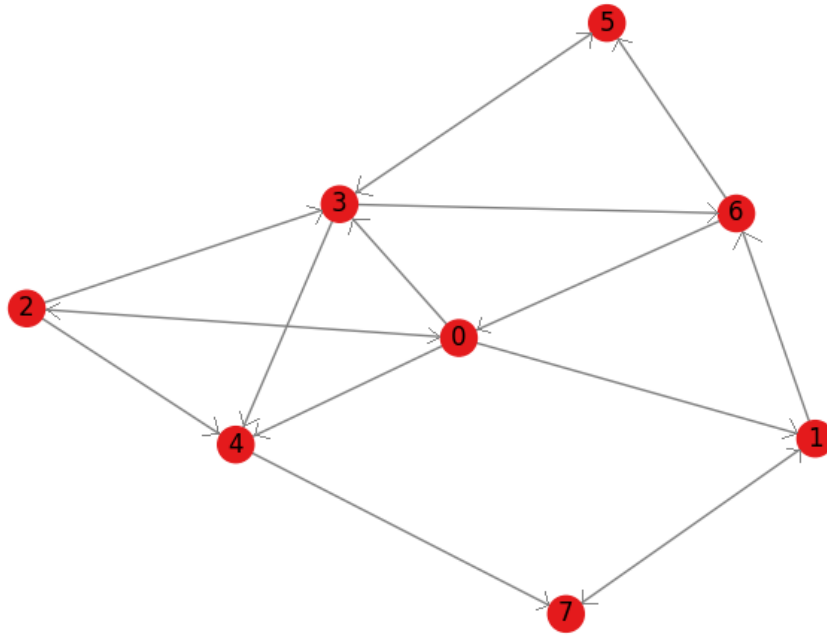
**Παράδειγμα 1.1.** Για παράδειγμα, ο παρακάτω μη κατευθυνόμενος γράφος έχει πίνακα γειτνίασης:



Σχήμα 1: Παράδειγμα απεικόνισης μη κατευθυνόμενου γράφου.

$$A_1 = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}. \quad (1.1)$$

Αν είχαμε κατευθυνόμενο γράφο όπως στο παρακάτω σχήμα, τότε ο πίνακας γειτνίασης θα ήταν:



Σχήμα 1.2: Παράδειγμα απεικόνισης κατευθυνόμενου γράφου.

$$A_2 = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Παρατηρούμε ότι οι δυο πίνακες δεν είναι ίδιοι και συγκεκριμένα ο  $A_2$  δεν είναι συμμετρικός. Στην παρούσα εργασία, θα ασχοληθούμε μόνο με μη κατευθυνόμενους γράφους.

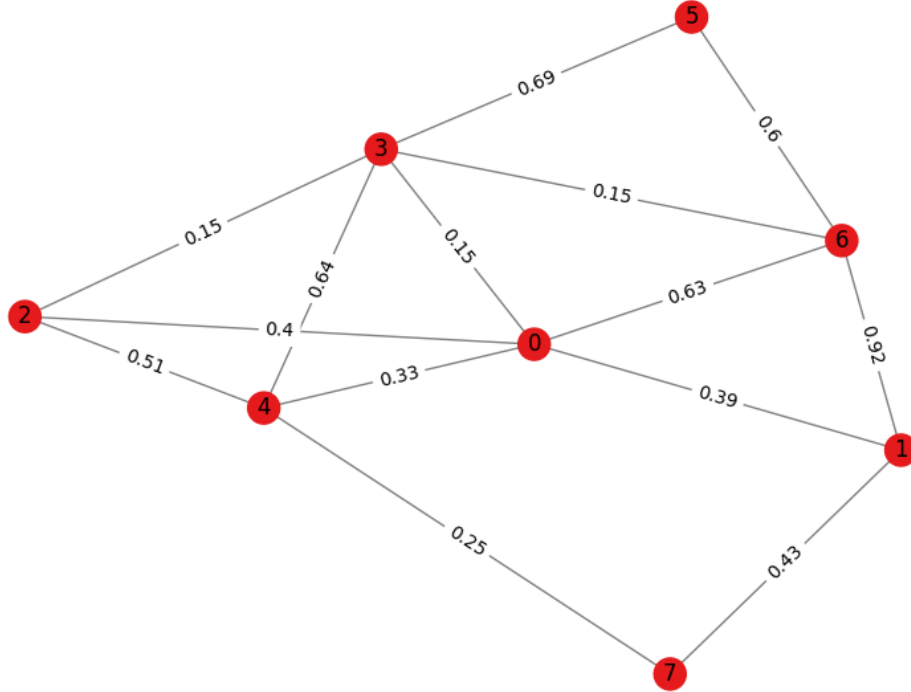
Εάν σε έναν γράφο έχουμε βάρη, τότε γράφουμε:

$$W_{ij} = \begin{cases} w_{i,j}, & (i,j) \in E \\ 0, & (i,j) \notin E \end{cases}$$

και μιλάμε για σταθμισμένο πίνακα βαρών. Το  $w_{ij}$  μπορεί να πάρει οποιαδήποτε θετική τιμή. Προφανώς, εάν  $w_{i,j} = 1$ , τότε

$$W = A.$$

**Παράδειγμα 1.2.** Αν ο πίνακας γειτνίασης έχει βάρη όπως στο ακόλουθο σχήμα



Σχήμα 1.3: Παράδειγμα απεικόνισης γράφου με βάρη στις ακμές.

τότε ο πίνακας βαρών είναι ο εξής:

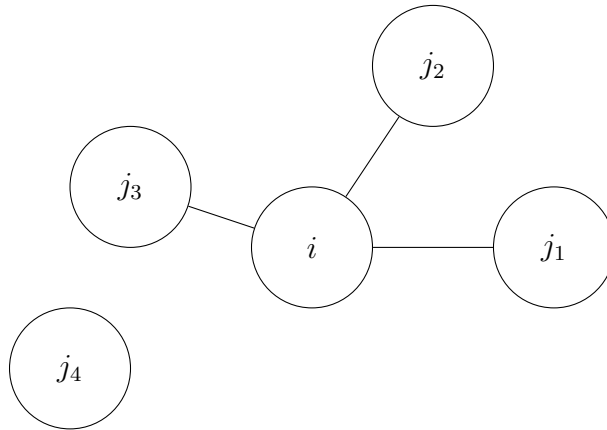
$$W = \begin{bmatrix} 0 & 0.39 & 0.4 & 0.15 & 0.33 & 0 & 0.63 & 0 \\ 0.39 & 0 & 0 & 0 & 0 & 0 & 0.92 & 0.43 \\ 0.4 & 0 & 0 & 0.15 & 0.51 & 0 & 0 & 0 \\ 0.15 & 0 & 0.15 & 0 & 0.64 & 0.69 & 0.15 & 0 \\ 0.33 & 0 & 0.51 & 0.64 & 0 & 0 & 0 & 0.25 \\ 0 & 0 & 0 & 0.69 & 0 & 0 & 0.6 & 0 \\ 0.63 & 0.92 & 0 & 0.15 & 0 & 0.6 & 0 & 0 \\ 0 & 0.43 & 0 & 0 & 0.25 & 0 & 0 & 0 \end{bmatrix}. \quad (1.2)$$

**Ορισμός 1.3.** Καλούμε **γειτονιά**  $N(i)$  του κόμβου  $i \in V$  ενός μη κατευθυνόμενου γράφου  $G = (V, E)$ , το σύνολο των κόμβων που συνδέονται με τον κόμβο  $i$ .

$$N(i) = \{j \in V \mid (i, j) \in E\}.$$

Ονομάζουμε βαθμό κόμβου  $v$  σε έναν μη κατευθυνόμενο γράφο να είναι ο αριθμός των ακμών που συνδέονται με τον κόμβο αυτό και συμβολίζεται ως  $\deg(v)$ .

**Παράδειγμα 1.3.** Παραθέτουμε παρακάτω ένα παράδειγμα γειτονιάς του κόμβου  $i$ .



Σχήμα 1.4: Γειτονιά κόμβου  $i$ .

Η γειτονιά του κόμβου  $i$  είναι το σύνολο  $N(i) = \{j_1, j_2, j_3\}$ . Ο κόμβος  $j_4$  δεν ανήκει στη γειτονιά, καθώς δεν συνδέεται με τον  $i$ . Ο βαθμός του  $i$  είναι  $\deg(i) = 3$ .

**Ορισμός 1.4.** Καλούμε πίνακα βαθμού να είναι ο διαγώνιος πίνακας

$$D = \begin{pmatrix} d_{1,1} & 0 & \cdots & 0 \\ 0 & d_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_{N,N} \end{pmatrix},$$

με στοιχεία

$$d_{i,i} = \sum_{j=1}^N w_{i,j},$$

όπου  $w_{i,j}$  τα βάρη στην  $i$ -γραμμή του πίνακα βαρών  $W$ .

Αν πάρουμε τον πίνακα  $W$  όπως στην (1.2), τότε:

$$D = \begin{bmatrix} 1.9 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1.74 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1.06 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.78 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1.73 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1.29 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2.3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.68 \end{bmatrix}.$$

**Ορισμός 1.5.** Για τον παραπάνω πίνακα βαρών  $W$  και τον πίνακα βαθμού  $D$  που αντιστοιχούν σε γράφο  $G$ , ορίζουμε τη Λαπλασιανή του  $G$  ως εξής:

$$L = D - W. \quad (1.3)$$

Για παράδειγμα, εάν ο πίνακας  $W$  είναι όπως στην (1.2), τότε:

$$L = \begin{bmatrix} 1.9 & -0.39 & -0.4 & -0.15 & -0.33 & 0 & -0.63 & 0 \\ -0.39 & 1.74 & 0 & 0 & 0 & 0 & -0.92 & -0.43 \\ -0.4 & 0 & 1.06 & -0.15 & -0.51 & 0 & 0 & 0 \\ -0.15 & 0 & -0.15 & 1.78 & -0.64 & -0.69 & -0.15 & 0 \\ -0.33 & 0 & -0.51 & -0.64 & 1.73 & 0 & 0 & -0.25 \\ 0 & 0 & 0 & -0.69 & 0 & 1.29 & -0.6 & 0 \\ -0.63 & -0.92 & 0 & -0.15 & 0 & -0.6 & 2.3 & 0 \\ 0 & -0.43 & 0 & 0 & -0.25 & 0 & 0 & 0.68 \end{bmatrix}.$$

**Ιδιότητες της Λαπλασιανής για μη κατευθυνόμενους γράφους**

1. Για κάθε διάνυσμα (ή πίνακα στήλη)  $f = (f_1, \dots, f_n) \in \mathbb{R}^n$  έχουμε:

$$f^* L f = \frac{1}{2} \sum_{i,j=1}^n w_{i,j} (f_i - f_j)^2.$$

2. Ο πίνακας  $L$  είναι συμμετρικός και θετικά ημιορισμένος.
3. Η μικρότερη ιδιοτιμή του  $L$  είναι μηδέν και αντιστοιχεί στο σταθερό ιδιοδιάνυσμα  $\vec{1} = (1, \dots, 1)$ .
4. Ο  $L$  έχει μη αρνητικές ιδιοτιμές

$$0 \leq \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n.$$

**Απόδειξη**

$$\begin{aligned} f^* L f &= f^* D f - f^* W f = \sum_{i=1}^n d_i f_i^2 - \sum_{i,j=1}^n f_i f_j w_{ij} \\ &= \frac{1}{2} \left( \sum_{i=1}^n d_i f_i^2 - 2 \sum_{i,j=1}^n f_i f_j w_{ij} + \sum_{j=1}^n d_j f_j^2 \right) = \frac{1}{2} \sum_{i,j=1}^n w_{ij} (f_i - f_j)^2. \end{aligned}$$

□

Αν  $D, W$  είναι όπως παραπάνω, τότε μπορούμε να ορίσουμε και την κανονικοποιημένη Λαπλασιανή ως εξής:

$$L_N = D^{-\frac{1}{2}} L D^{-\frac{1}{2}} = D^{-\frac{1}{2}} (D - W) D^{-\frac{1}{2}}.$$

Ετσι, για τους παραπάνω πίνακες  $D, W$  από την (1.2), παίρνουμε

$$L_N = \begin{bmatrix} 1 & -0.214 & -0.282 & -0.08 & -0.182 & 0 & -0.301 & 0 \\ -0.214 & 1 & 0 & 0 & 0 & 0 & -0.46 & -0.395 \\ -0.282 & 0 & 1 & -0.109 & -0.376 & 0 & 0 & 0 \\ -0.08 & 0 & -0.109 & 1 & -0.364 & -0.455 & -0.07 & 0 \\ -0.182 & 0 & -0.376 & -0.364 & 1 & 0 & 0 & -0.23 \\ 0 & 0 & 0 & -0.455 & 0 & 1 & -0.348 & 0 \\ -0.301 & -0.46 & 0 & -0.07 & 0 & -0.348 & 1 & 0 \\ 0 & -0.395 & 0 & 0 & -0.23 & 0 & 0 & 1 \end{bmatrix}.$$

Σημειώνουμε ότι για κάθε διάνυσμα  $f \in \mathbb{R}^n$  έχουμε:

$$f^* L_N f = \frac{1}{2} \sum_{i,j=1}^n w_{i,j} \left( \frac{f_i}{\sqrt{d_i}} - \frac{f_j}{\sqrt{d_j}} \right)^2.$$

### Παρατηρήσεις

1. Σε μη κατευθυνόμενους γράφους, οι πίνακες  $A, D, W$  είναι συμμετρικοί.
2. Η Λαπλασιανή δεν εξαρτάται από τα στοιχεία της κύριας διαγωνίου του πίνακα βαρών  $W$ .
3. Διαδρομή ή μονοπάτι μήκους  $k$  είναι μια ακολουθία κόμβων  $(u_i)_{i=1}^k$  του γράφου  $G = (V, E)$ , εάν  $(u_{i-1}, u_i) \in E$  για κάθε  $i = 1, 2, \dots, k$  και δεν υπάρχει κάποιος κόμβος που να εμφανίζεται πάνω από μια φορά.
4. Η απόσταση  $d(u, v)$  μεταξύ 2 κόμβων  $u$  και  $v$  ισούται με το ελάχιστο μήκος όλων των διαδρομών ανάμεσα τους.
5. Η διάμετρος ενός συνεκτικού γράφου ορίζεται ως η μέγιστη απόσταση μεταξύ 2 οποιoδήποτε κόμβων  $u$  και  $v$ :

$$d(G) = \max \{d(u, v) \mid u, v \in V(G)\}.$$

## 1.2 Φασματική Ανάλυση του Πίνακα Γειτνίασης

Οι γράφοι μπορούν να αναλυθούν με την χρήση των ιδιοτιμών του πίνακα γειτνίασης. Υπενθυμίζουμε ότι ένα μη μηδενικό διάνυσμα  $u$  καλείται ιδιοδιάνυσμα ενός τετραγωνικού πίνακα  $A$ , που αντιστοιχεί στην ιδιοτιμή  $\lambda$ , εάν:

$$A\vec{u} = \lambda\vec{u}.$$

Η παραπάνω γράφεται και ως:

$$(A - \lambda I)\vec{u} = 0 \Rightarrow |A - \lambda I| = 0,$$

οπότε προκύπτει το χαρακτηριστικό πολυώνυμο

$$P = |A - \lambda I| = \lambda^N + c_1 \lambda^{N-1} + \dots + c_N.$$

Οι ιδιοτιμές του πίνακα  $A$  είναι οι ρίζες της εξίσωσης

$$P(\lambda) = 0.$$

Αρα, αν  $\lambda_1, \dots, \lambda_N$  οι ιδιοτιμές του  $A$ , τότε:

$$P(\lambda) = (\lambda - \lambda_1) \dots (\lambda - \lambda_N)$$

και για  $\lambda = 0$  προκύπτει ότι

$$P(0) = (-\lambda_1)(-\lambda_2) \dots (-\lambda_N).$$

### Παρατηρήσεις

1. Η τάξη του πολυωνύμου  $P$  ισούται με τον αριθμό των κορυφών  $N$  του γράφου.
2. Το σύνολο των ιδιοτιμών  $\lambda$  του πίνακα γειτνίασης ονομάζεται φάσμα γειτνίασης του σχετικού γράφου.

**Θεώρημα 1.6.** [2] Αν τα ιδιοδιανύσματα τετραγωνικού πίνακα  $A$  είναι γραμμικώς ανεξάρτητα τότε ο πίνακας  $A$  διαγωνοποιείται ως εξής:

$$A = U\Lambda U^{-1},$$

όπου  $\Lambda$  διαγώνιος πίνακας με διαγώνια στοιχεία τις ιδιοτιμές του  $A$  και  $U$  αντιστρέψιμος πίνακας με στήλες τα ιδιοδιανύσματα του πίνακα  $A$ .

**Ορισμός 1.7.** Ένας πραγματικός (ή μιγαδικός) πίνακας ονομάζεται συμμετρικός (ή Ερμητιανός), αν:

$$A^T = A \text{ (ή } A^* = A.)$$

**Θεώρημα 1.8.** [2] Οι ιδιοτιμές ενός συμμετρικού (ή Ερμητιανού) πίνακα  $A$  είναι πραγματικές και τα ιδιοδιανύσματά του είναι ανά δύο ορθογώνια. Έτσι έχουμε μια διαγωνοποίηση του πίνακα αυτού ως εξής:

$$A = U\Lambda U^T \text{ (ή } A = U\Lambda U^*)$$

όπου  $U$  ορθομοναδιαίος πίνακας ιδιοδιανυσμάτων του  $A$ .

Αντίστοιχα με τον πίνακα γειτνίασης  $A$ , μπορούμε από την (1.3) να κάνουμε φασματική ανάλυση της Λαπλασιανής. Ισχύουν οι ίδιες εξισώσεις:

$$L\vec{u}_k = \lambda_k \vec{u}_k$$

και

$$L = U\Lambda'U^T = U(D - \Lambda)U^T.$$

**Παράδειγμα 1.4.** Έστω ότι έχουμε τον παρακάτω πίνακα γειτνίασης  $A$  για έναν γράφο 3 κορυφών:

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

Θέλουμε να διαγωνοποιήσουμε τον πίνακα  $A$ , δηλαδή να βρούμε έναν ορθομοναδιαίο πίνακα  $U$  και διαγώνιο πίνακα  $\Lambda$  ώστε:

$$A = U\Lambda U^\top.$$

Βρίσκουμε πρώτα τις ιδιοτιμές, λύνοντας την εξίσωση:

$$\det(A - \lambda I) = 0.$$

Έχουμε:

$$\begin{aligned} A - \lambda I &= \begin{bmatrix} -\lambda & 1 & 1 \\ 1 & -\lambda & 1 \\ 1 & 1 & -\lambda \end{bmatrix} \\ \Rightarrow P(\lambda) = \det(A - \lambda I) &= -\lambda^3 + 3\lambda - 2 = 0. \end{aligned}$$

Οι ρίζες αυτής της εξίσωσης είναι  $\lambda_1 = 2$ ,  $\lambda_2 = -1$ ,  $\lambda_3 = -1$ . Για κάθε ιδιοτιμή λύνουμε το σύστημα  $(A - \lambda I)\vec{u} = 0$  και παίρνουμε τα ιδιοδιανύσματα.

- Για  $\lambda = 2$ , βρίσκουμε το ιδιοδιάνυσμα  $\vec{u}_1 = \frac{1}{\sqrt{3}}[1, 1, 1]^\top$ .
- Για  $\lambda = -1$ , βρίσκουμε δύο γραμμικά ανεξάρτητα ιδιοδιανύσματα:

$$\vec{u}_2 = \frac{1}{\sqrt{2}}[1, -1, 0]^\top, \quad \vec{u}_3 = \frac{1}{\sqrt{6}}[1, 1, -2]^\top.$$

Τέλος, κάνουμε διαγωνοποίηση και παίρνουμε τον πίνακα ιδιοδιανυσμάτων:

$$U = [\vec{u}_1 \ \vec{u}_2 \ \vec{u}_3] = \begin{bmatrix} \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & 0 & -\frac{2}{\sqrt{6}} \end{bmatrix},$$

με το διαγώνιο πίνακα ιδιοτιμών να είναι ο εξής:

$$\Lambda = \begin{bmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

Με τον ίδιο ακριβώς τρόπο θα κάνουμε φασματική ανάλυση της Λαπλασιανής:

$$L = D - A = \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}.$$

Λύνουμε την εξίσωση:

$$\det(L - \lambda I) = 0.$$

Βρίσκουμε:

$$\begin{aligned} L - \lambda I &= \begin{bmatrix} 2 - \lambda & -1 & -1 \\ -1 & 2 - \lambda & -1 \\ -1 & -1 & 2 - \lambda \end{bmatrix} \\ \Rightarrow P(\lambda) = \det(L - \lambda I) &= \lambda^3 - 6\lambda^2 + 9\lambda = 0. \end{aligned}$$

Οι ρίζες αυτής της εξίσωσης είναι  $\lambda_1 = 0$ ,  $\lambda_2 = 3$ ,  $\lambda_3 = 3$ . Προκύπτει ο ίδιος ακριβώς πίνακας ιδιοδιανυσμάτων  $U$ , όπως στην (1.4), αλλάζει όμως ο πίνακας ιδιοτιμών, σε

$$\Lambda' = D - \Lambda = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

Επαληθεύεται ότι:

$$A = U\Lambda U^\top \text{ και } L = U\Lambda' U^\top.$$

## 2 Φασματική συσταδοποίηση γράφων

Στο κεφάλαιο αυτό μελετούμε δύο βασικούς αλγορίθμους φασματικής συσταδοποίησης γράφων: τον αλγόριθμο φασματικής συσταδοποίησης Spectral Clustering και τον αλγόριθμο κανονικοποιημένης τομής Normalized Cut. Και οι δύο μέθοδοι χρησιμοποιούν τη φασματική ανάλυση του γραφήματος, δηλαδή την ανάλυση ιδιοτιμών και ιδιοδιανυσμάτων της Λαπλασιανής, προκειμένου να διαμερίσουν (σε μη κενά ξένα μεταξύ τους υποσύνολα κόμβων) το γράφο.

Η φασματική συσταδοποίηση ανάγεται στους Donath, Homan (1973), οι οποίοι ήταν οι πρώτοι που πρότειναν την κατασκευή διαμέρισης γραφημάτων με βάση τα ιδιοδιανύσματα του πίνακα γειτνίασης. Την ίδια χρονιά, ο Fiedler (1973) ανακάλυψε ότι η δυαδική διαμέριση ενός γραφήματος συνδέεται στενά με το μικρότερο μη μηδενικό ιδιοδιάνυσμα της Λαπλασιανής και πρότεινε την κατάλληλη χρήση αυτού του ιδιοδιανύσματος για τη διαμέριση ενός γραφήματος.

Οι φασματικοί αλγόριθμοι διαμέρισης έχουν αποκτήσει μεγάλη δημοτικότητα στην κοινότητα της μηχανικής μάθησης, λόγω της ικανότητάς τους να εντοπίζουν σύνθετες δομές σε δεδομένα. Η φασματική συσταδοποίηση έγινε δημοφιλής χάρη στις εργασίες [5] (2000), Ng et al. (2002), και Meila και Shi (2001), Ding (2004), συνδέοντας την με το πρόβλημα ελαχιστοποίησης του Normalized Cut συμβάλλοντας στην κατανόηση του διαχωρισμού των δεδομένων με βάση τη δομή γειτνίασης.

### 2.1 Αλγόριθμος Spectral Clustering

Στην παράγραφο αυτή, εξετάζουμε τον αλγόριθμο φασματικής συσταδοποίησης (Spectral Clustering), βλέπε [3], ο οποίος διαμερίζει έναν γράφο σε ομάδες κόμβων με βάση τις φασματικές ιδιότητες της Λαπλασιανής του γράφου. Η κύρια ιδέα πίσω από τον αλγόριθμο είναι η εξής: οι ιδιοτιμές και τα ιδιοδιανύσματα του πίνακα της Λαπλασιανής περιέχουν πληροφορία για τη δομή του γράφου, και πιο συγκεκριμένα για την τοπολογία των συνδέσεων μεταξύ των κόμβων. Χρησιμοποιώντας τη φασματική πληροφορία και τον αλγόριθμο k-means, είναι δυνατό να ομαδοποιηθούν οι κόμβοι σε συστάδες όπου οι κόμβοι είναι περισσότερο συνδεδεμένοι μεταξύ τους, απ' ό,τι με κόμβους άλλων συστάδων. Η γενική μεθοδολογία του αλγορίθμου περιλαμβάνει τα εξής βήματα:

- Κατασκευή πίνακα βαρών  $W$  και διαγώνιου πίνακα βαθμών  $D$ .
- Υπολογισμός του Λαπλασιανού πίνακα  $L = D - W$ .
- Εξαγωγή των ιδιοδιανυσμάτων που αντιστοιχούν στις  $k$  μικρότερες θετικές ιδιοτιμές του  $L$ .
- Εφαρμογή του αλγορίθμου k-means στις γραμμές του πίνακα  $N \times k$  ιδιοδιανυσμάτων.

Πριν προχωρήσουμε, θεωρούμε χρήσιμο να υπενθυμίσουμε συνοπτικά τον αλγόριθμο k-means, ο οποίος χρησιμοποιείται στο τελικό στάδιο του Spectral Clustering. Ο αλγόριθμος k-means αποτελεί μια απλή και αποτελεσματική μέθοδο

συσταδοποίησης. Έχει ως στόχο να χωρίσει ένα σύνολο δεδομένων σε  $k$  συστάδες, ελαχιστοποιώντας την Ευκλείδεια αποστάσεων μεταξύ των σημείων από τα κέντρα των συστάδων τους. Ωστόσο, για περίπλοκα σχήματα και κατανομές ο αλγόριθμός δεν είναι τόσο αποτελεσματικός. Χρησιμοποιείται ευρέως στην ανάλυση-εξόρυξη δεδομένων, στην αναγνώριση προτύπων, στη συμπίεση εικόνας και στη μηχανική μάθηση.

Ο ψευδοκώδικας του αλγορίθμου  $k$ -means (μαζί με την αρχικοποίηση):

---

### Αλγόριθμος 1 $k$ -means

---

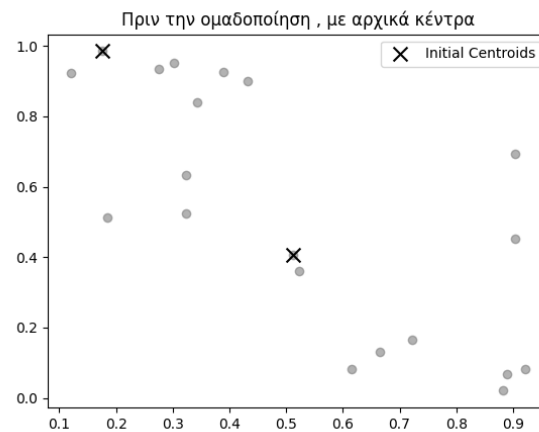
```

1: Είσοδος: Σύνολο σημείων  $P = (p_i)_{i=1}^n \subset \mathbb{R}^k$ , ανοχή  $\varepsilon > 0$ .
2:  $P_1 = \emptyset, P_2 = \emptyset, \dots, P_k = \emptyset$ .
3: Διαλέγω τυχαία  $k$  σημεία  $c_1, c_2, \dots, c_k$  ως κέντρα των  $P_1, P_2, \dots, P_k$ .
4: while  $\|c_i - c'_i\| > \varepsilon$  do
5: Για κάθε  $p \in P$ , υπολόγισε:
6:  $i_0 = \arg \min_{i=1, \dots, k} \|p - c_i\|^2$ .
7:  $P_i \leftarrow P_i \cup \{p\}$ .
8: for  $i = 1$  to  $k$  do
9:  $c'_i = \frac{1}{|P_i|} \sum_{p \in P_i} p$ .
10: end for
11: Υπολόγισε  $\|c_i - c'_i\|$ .
12: end while
13: return  $P_1, \dots, P_k$ 

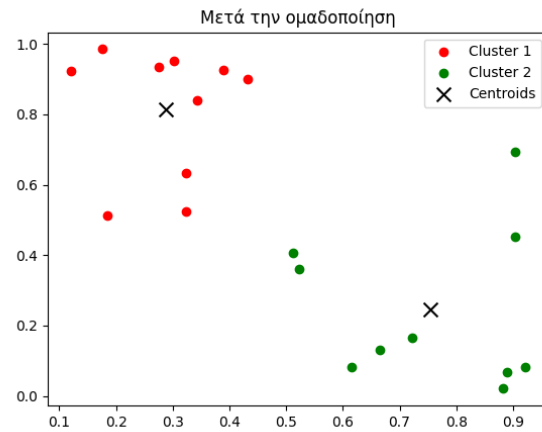
```

---

**Παράδειγμα 2.1.** Παράδειγμα συσταδοποίησης 20 σημείων με 2 αρχικά κέντρα:

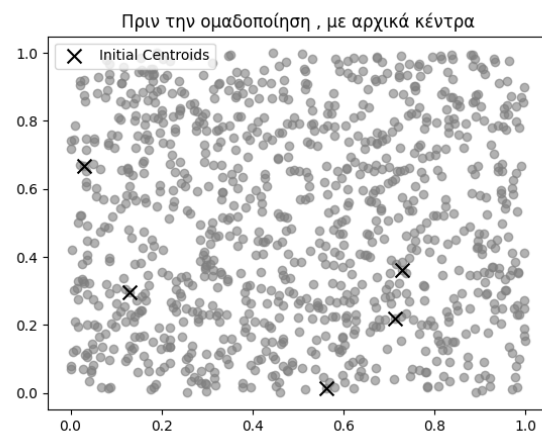


Σχήμα 2.1: Αρχικοποίηση κέντρων.

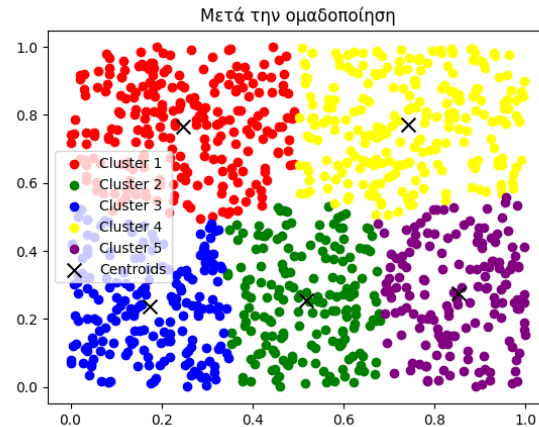


Σχήμα 2.2: Δημιουργία συστάδων.

**Παράδειγμα 2.2.** Παράδειγμα συσταδοποίησης 1000 σημείων με 5 αρχικά κέντρα:



Σχήμα 2.3: Αρχικοποίηση κέντρων.



Σχήμα 2.4: Δημιουργία συστάδων.

Στην παρούσα εφαρμογή, ο αλγόριθμος  $k$ -means εφαρμόζεται όχι στα αρχικά δεδομένα, αλλά στις γραμμές του πίνακα  $N \times k$  ιδιοδιανυσμάτων που περιγράφουν τη φασματική δομή του γράφου. Ο ψευδοκώδικας του αλγορίθμου  $k$ -means παρατίθεται παραπάνω, με έμφαση στην αρχικοποίηση που απαιτείται ώστε να εφαρμοστεί σωστά στο πλαίσιο του Spectral Clustering.

Τώρα είμαστε έτοιμοι να περιγράψουμε τον αλγόριθμο φασματικής συσταδοποίησης, ένα βασικό συστατικό του οποίου είναι ο αλγόριθμος  $k$ -means.

---

## Αλγόριθμος 2 Αλγόριθμος Spectral Clustering

---

- 1: Έστω γράφος με  $N$  κόμβους και  $M$  ακμές.
  - 2: Υπολογίζουμε τον πίνακα γειτνίασης  $A$ .
  - 3: Υπολογίζουμε την Λαπλασιανή  $L = D - A$ .
  - 4: Υπολογίζουμε τις  $k$ -μικρότερες θετικές ιδιοτιμές σε αύξουσα σειρά μαζί με τα αντίστοιχα ιδιοδιανύσματα.
  - 5: Βάζουμε τα  $k$ -ιδιοδιανύσματα ως στήλες ενός πίνακα διάστασης  $N \times k$ .
  - 6: Εφαρμόζουμε τον αλγόριθμο  $k$  – *means* για τις γραμμές του πίνακα  $N \times k$  του βήματος 5, τις οποίες θεωρούμε ως σημεία του συνόλου  $P \in \mathbb{R}^k$ .
- 

Παραθέτουμε τον κώδικα σε python.

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Βήμα 1: Δημιουργία τυχαίου γράφου
N = 8 # number of nodes
M = 14 # number of edges
G = nx.gnm_random_graph(n=N, m=M, seed=42)
```

```

# Βήμα 2: Πίνακας γειτνίασης και Λαπλασιανή του γράφου
A = nx.to_numpy_array(G, dtype=int)
degrees = np.sum(A, axis=1)
D = np.diag(degrees)
L = D - A

# Βήμα 3: Αποσύνθεση ιδιοτιμών/ιδιοδιανυσμάτων
eigenvalues, eigenvectors = np.linalg.eigh(L)
k = 2
top_k_eigenvectors = eigenvectors[:, :k]
top_k_eigenvalues = eigenvalues[:k]

# Βήμα 4: KMeans για τις γραμμές του πίνακα Nxk
X = top_k_eigenvectors # shape: (N, k)
kmeans = KMeans(n_clusters=k, n_init=1500, random_state=0).fit(X)
labels = kmeans.labels_

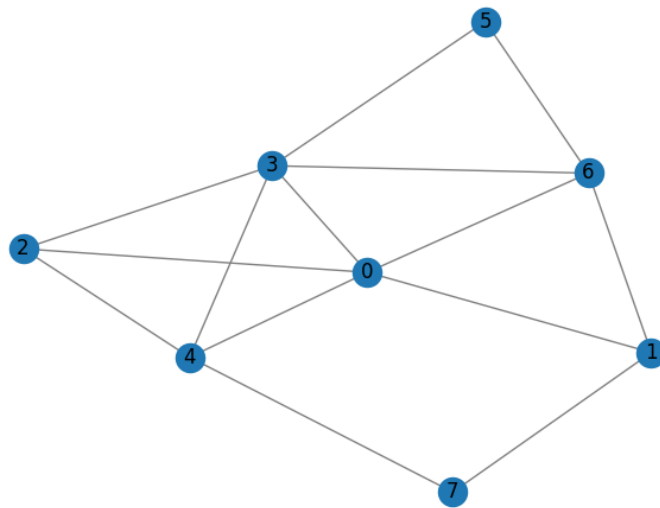
# Βήμα 5: Έξοδος
print("Cluster assignments for each node:", labels)
print("Laplacian matrix L:")
print(L)
print("Top k eigenvalues:", top_k_eigenvalues)
print("Top k eigenvectors:", top_k_eigenvectors)

# Βήμα 6: Σχήματα γράφων
nx.draw(G, with_labels=True, node_color=labels, cmap=plt.cm.Set1, edge_color='grey',
plt.show()

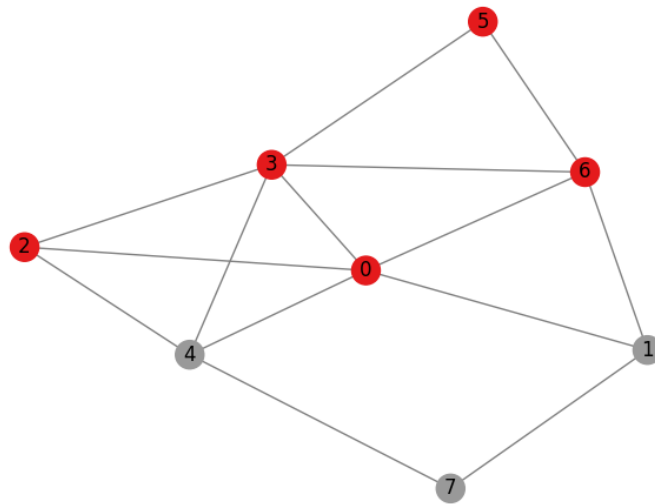
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_color=labels, cmap=plt.cm.Set1, edge_color='grey',
plt.show()

```

**Παράδειγμα 2.3.** Παρακάτω παραθέτουμε μια εικόνα ενός γράφου που έχει 8 κόμβους και 14 ακμές. Θέτοντας  $k = 2$  βρίσκουμε μια διαμέριση του γράφου σε 2 συστάδες.



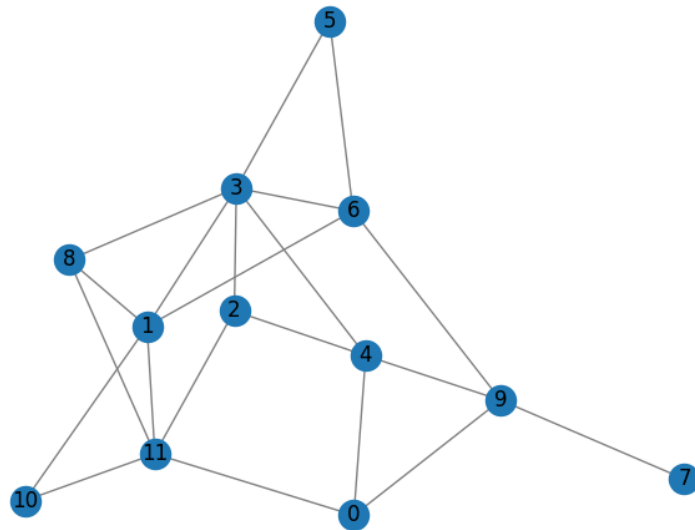
Σχήμα 2.5: Αρχικός γράφος.



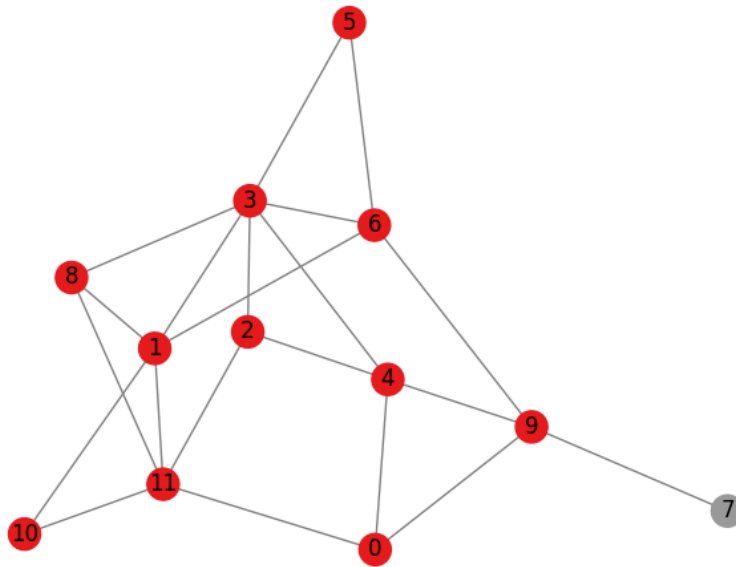
Σχήμα 2.6: Τελικός γράφος.

Βλέπουμε ότι ο αλγόριθμος διαμερίζει το γράφο σε δύο ομάδες, τις εξής:  $A = \{0, 2, 3, 5, 6\}$  και  $B = \{1, 4, 7\}$ .

**Παράδειγμα 2.4.** Παρακάτω παραθέτουμε μια εικόνα ενός γράφου που έχει 12 κόμβους και 21 ακμές. Θέτοντας το  $k = 2$  βρίσκουμε μια διαμέριση του γράφου σε 2 συστάδες.

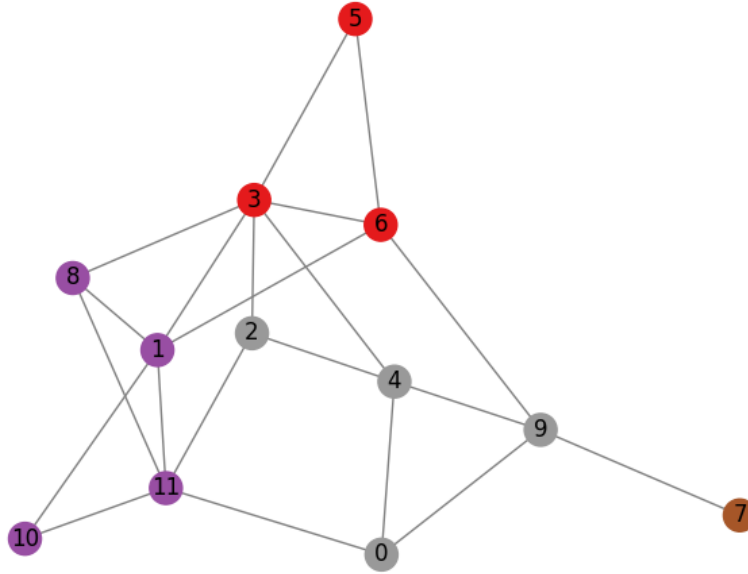


Σχήμα 2.7: Αρχικός γράφος.



Σχήμα 2.8: Τελικός γράφος.

Παρατηρούμε ότι για  $k = 2$  η διαμέριση δεν είναι ισοσταθμισμένη και η μια ομάδα αποτελείται από έναν μόνο κομβό, το οποίο είναι ένα σημαντικό μειονέκτημα της μεθόδου. Από την άλλη μπορούμε να αυξήσουμε τον αριθμό των ομάδων για να γίνει πιο ισορροπημένη η διαμέριση.



Σχήμα 2.9: Τελικός γράφος σε 4 συστάδες.

## 2.2 Αλγόριθμοι Graph Cut

Το 1938 ο Wertheimer επεσήμανε τη σημασία της αντιληπτικής ομαδοποίησης στην όραση και απαρίθμησε αρκετούς βασικούς παράγοντες, όπως η ομοιότητα και η εγγύτητα που οδηγούν στην οπτική ομαδοποίηση. Για να αντιμετωπιστούν πολλά από τα υπολογιστικά ζητήματα της αντιληπτικής ομαδοποίησης αναπτύχθηκε αρχικά ο αλγόριθμος τομής γράφου (graph cut). Δεδομένου ότι υπάρχουν πολλές πιθανές διαμερίσεις/ομαδοποιήσεις σε μια εικόνα σε υποσύνολα, πώς επιλέγουμε τη σωστή; Υπάρχουν δύο πτυχές που πρέπει να ληφθούν υπόψη εδώ. Η πρώτη είναι ότι μπορεί να μην υπάρχει μία μόνο σωστή απάντηση. Η δεύτερη πτυχή είναι ότι η διαμέριση παρουσιάζει μια εγγενώς ιεραρχική δομή. Επομένως, είναι προτιμότερο να προσεγγίσουμε το πρόβλημα μέσω μιας δενδρωτής δομής, η οποία αναπαριστά τη διαμέριση με ιεραρχικό τρόπο αντί για μια ενιαία «επίπεδη» διαμέριση. Συνεπώς, η τμηματοποίηση εικόνας δεν αποτελεί μια διαδικασία που πρέπει να στοχεύει άμεσα στην παραγωγή μιας πλήρους και τελικής «σωστής» διαμέρισης. Αντίθετα, είναι πιο λογικό να γίνεται σταδιακά, ξεκινώντας από τις μεγάλες και βασικές περιοχές και στη συνέχεια να προχωράει προς τις λεπτομέρεις. Με αυτόν τον τρόπο, μπορούμε να αποτυπώσουμε καλύτερα τη δομή της εικόνας και να εντοπίσουμε τις σημαντικές πληροφορίες σε διάφορα επίπεδα. Η προσέγγιση αυτής της ιεραρχικής δομής παράγει ένα δέντρο. Στη συγκεκριμένη εργασία, βασιζόμαστε στην θεωρία γράφων με εφαρμογή στη διαμέριση γράφων. Στόχος είναι να διαμεριστεί το γράφημα σε υποσύνολα κόμβων με υψηλή εσωτερική συνοχή και χαμηλή συσχέτιση μεταξύ διαφορετικών ομάδων με βάση δυο σημαντικά ζητήματα: Πρώτον, το ακριβές κριτήριο διαμέρισης και δεύτερον, τον αποτελεσματικό υπολογισμό της διαμέρισης. Για την παράγραφο αυτή παραπέμπουμε στο [5].

Έστω ότι ο γράφος  $G = (V, E)$  διαμερίζεται σε σύνολα  $A$  και  $B$  με  $G = A \cup B$

και  $A \cap B = \emptyset$ , πολύ απλά απομονώνοντας (αφαιρώντας) όλες τις ακμές απο το  $A$  προς το  $B$  και αντιστρόφως.

**Ορισμός 2.1.** Ο βαθμός ανομοιότητας μεταξύ των συνόλων  $A$  και  $B$  ορίζεται ως το συνολικό βάρος των ακμών που έχουν αποκοπεί από τα  $A$  και  $B$ . Αυτό το βαθμό καλούμε τομή (*cut*) και τον συμβολίζουμε με

$$cut(A, B) = \sum_{\substack{u \in A \\ v \in B}} w(u, v).$$

Η βέλτιστη διαμέριση του γράφου σε δυο σύνολα  $A$  και  $B$  προκύπτει απο την ελαχιστοποίηση αυτής της ποσότητας. Παρότι υπάρχει εκθετικό πλήθος τέτοιων διαμερίσεων, η εύρεση των βέλτιστων κοψιμάτων είναι ένα καλά μελετημένο πρόβλημα και υπάρχουν αλγόριθμοι για τη λύση του.

Οι Wu και Leahy πρότειναν μια μέθοδο συσταδοποίησης χρησιμοποιώντας το κριτήριο συσταδοποίησης *cut*. Πιο συγκεκριμένα, έψαξαν, με παρόμοια λογική όπως παραπάνω να επιτύχουν διαμέριση του γράφου σε  $k$  - κομμάτια ώστε το κόστος  $cut(A, B)$  να ελαχιστοποιείται ως προς όλα τα ζεύγη  $(A, B)$ . Αλλά παρατήρησαν ότι η μέθοδος αυτή οδηγεί σε διαμερίσεις του γράφου σε μικρά συνήθως σύνολα που περιείχαν τους πιο απομονωμένους κόμβους του γράφου. Για να αποφύγουνε αυτή την τάση προτείναν την κανονικοποιημένη τομή (*normalized cut*), η οποία ορίζεται ως εξής:

**Ορισμός 2.2.** Καλούμε ως κανονικοποιημένη τομή γράφου  $G$  σε σύνολα  $A$  και  $B$  να είναι ο θετικός αριθμός

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(B, A)}{assoc(B, V)},$$

όπου

$$assoc(A, V) = \sum_{\substack{u \in A \\ t \in V}} w(u, t)$$

δηλώνει την ολική συνδεσιμότητα μεταξύ των κόμβων του συνόλου  $A$  ως προς τους κόμβους όλου του γράφου. Αντίστοιχα ορίζεται και το άθροισμα  $assoc(B, V)$ .

Με τον ορισμό αυτό, απομονωμένοι κόμβοι του γράφου δε θα έχουν μικρή τιμή  $Ncut$ , αφού η τιμή  $cut$  θα είναι ένα μεγάλο ποσοστό της ολικής συνδεσιμότητας απο το μικρό σύνολο στους άλλους κόμβους.

**Ορισμός 2.3.** Ομοίως ορίζουμε

$$Nassoc(A, B) = \frac{assoc(A, A)}{assoc(A, V)} + \frac{assoc(B, B)}{assoc(B, V)},$$

όπου  $assoc(A, A)$  και  $assoc(B, B)$  δηλώνουν τη συνολική συνδεσιμότητα των κόμβων στα σύνολα  $A$  και  $B$  αντιστοίχως.

Έχοντας ορίσει τα παραπάνω μπορούμε να δείξουμε ότι:

**Πρόταση 2.4.**

$$\frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)} = 2 - Nassoc(A, B).$$

**Απόδειξη**

$$\begin{aligned} Ncut &= \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)} \\ &= \frac{assoc(A, V) - assoc(A, A)}{assoc(A, V)} + \frac{assoc(B, V) - assoc(B, B)}{assoc(B, V)} \\ &= 2 - \left( \frac{assoc(A, A)}{assoc(A, V)} + \frac{assoc(B, B)}{assoc(B, V)} \right) = 2 - Nassoc(A, B). \end{aligned}$$

□

Δυστυχώς το να ελαχιστοποιήσουμε το  $Ncut$  είναι NP-πλήρες πρόβλημα. Παρόλα αυτά θα δείξουμε ότι αν ενσωματώσουμε το πρόβλημα από το διακριτό στο συνεχές πεδίο, προσεγγιστική λύση υπάρχει και είναι ικανοποιητική. Έστω διαμέριση των κόμβων γράφου  $G$  σε δυο σύνολα  $A$  και  $B$ , δηλαδή  $A \cup B = G$ ,  $A \cap B = \emptyset$  και έστω  $|\nu|$  το πλήθος των κόμβων του γράφου. Θεωρούμε  $\vec{\chi} \in \mathbb{R}^{|\nu|} : \vec{\chi} = (\chi_1, \chi_2, \dots, \chi_\nu)$ , έτσι ώστε

$$\chi_i = \begin{cases} 1, & \text{αν } i \in \text{κόμβος του } A \\ -1, & \text{αν } i \in \text{κόμβος του } B \end{cases}.$$

Εστω ο βαθμός

$$d(i) = \sum_{j=1}^{\nu} w(i, j),$$

δηλαδή ο αριθμός συνδέσμων του κόμβου  $i$  με όλους τους κόμβους του γράφου. Τώρα μπορούμε να ξαναγράψουμε

$$\begin{aligned} Ncut(A, B) &= \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(B, A)}{assoc(B, V)} \\ &= \frac{\sum_{(\chi_i > 0, \chi_j < 0)} -w_{ij} \chi_i \chi_j}{\sum_{\chi_i > 0} d_i} + \frac{\sum_{(\chi_i < 0, \chi_j > 0)} -w_{ij} \chi_i \chi_j}{\sum_{\chi_i < 0} d_i}. \end{aligned}$$

Έστω  $D$  ένας  $N \times N$  διαγώνιος πίνακας βαθμού όπως παραπάνω, με  $N = |\nu|$  και

$$W = (w_{i,j})_{N \times N}$$

συμμετρικός πίνακας βαρών του γράφου  $G$ . Ορίζουμε

$$k = \frac{\sum_{\chi_i > 0} d_i}{\sum_{i=1}^{|\nu|} d_i}$$

και

$$\vec{1} = (1, 1, \dots, 1)_{N \times 1}$$

διάνυσμα στήλη. Λαμβάνοντας υπόψη ότι τα διανύσματα

$$\frac{\vec{1} + \vec{\chi}}{2} \text{ και } \frac{\vec{1} - \vec{\chi}}{2}$$

είναι δείκτες, δηλαδή δείχνουν το φορέα των συνόλων  $A = \{\chi_i : \chi_i > 0\}$  και  $B(\text{όπου } \chi_i < 0)$  αντιστοίχως, έχουμε

$$Ncut(A, B) = \frac{(\vec{1} + \vec{\chi})^T \cdot (D - W) \cdot (\vec{1} + \vec{\chi})}{k (\vec{1}^T \cdot D \cdot \vec{1})} + \frac{(\vec{1} - \vec{\chi})^T \cdot (D - W) \cdot (\vec{1} - \vec{\chi})}{(1 - k) \vec{1}^T \cdot D \cdot \vec{1}}.$$

Κάνοντας πράξεις, αναλύουμε τους αριθμητές και παρονομαστές της παραπάνω σχέσης:

$$\bullet (\vec{1} + \vec{\chi})^T \cdot (D - W) \cdot (\vec{1} + \vec{\chi})$$

$$\begin{aligned} &= \sum_{\substack{\chi_i > 0 \\ \chi_j > 0}} \chi_i (d - W)_{ij} \chi_j = \sum_{\chi_i > 0} \chi_i d_i x_j - \sum_{\substack{\chi_i > 0 \\ \chi_j > 0}} \chi_i w_{ij} \chi_j \\ &= \sum_{\chi_i > 0} d_i - \sum_{\chi_i > 0} \chi_i (d_i - \sum_{\chi_j < 0} w_{ij}) (-\chi_j) = - \sum_{\substack{\chi_i > 0 \\ \chi_j < 0}} w_{ij} \chi_i \chi_j. \end{aligned}$$

$$\bullet (\vec{1} - \vec{\chi})^T \cdot (D - W) \cdot (\vec{1} - \vec{\chi})$$

$$\begin{aligned} &= \sum_{\substack{\chi_i < 0 \\ \chi_j < 0}} \chi_i (d - W)_{ij} \chi_j = \sum_{\chi_i < 0} \chi_i d_i x_j \\ &- \sum_{\chi_i < 0} \chi_i (d_i - \sum_{\chi_j > 0} w_{ij}) (-x_j) = - \sum_{\substack{\chi_i < 0 \\ \chi_j > 0}} \chi_i w_{ij} x_j. \end{aligned}$$

•

$$\sum_{\chi_i > 0} d_i = k \sum_{j=1}^{|\nu|} d_j = k (\vec{1} \cdot D \cdot \vec{1}).$$

•

$$\sum_{\chi_i < 0} d_i = (1 - k) \sum_{j=1}^{|\nu|} d_j = k (\vec{1} \cdot D \cdot \vec{1}).$$

Έτσι προκύπτει:

$$4Ncut(A, B) = \frac{\sum_{(\chi_i > 0, \chi_j < 0)} -w_{ij} \chi_i \chi_j}{\sum_{\chi_i > 0} d_i} + \frac{\sum_{(\chi_i < 0, \chi_j > 0)} -w_{ij} \chi_i \chi_j}{\sum_{\chi_i < 0} d_i}$$

$$+ \frac{(\vec{1} + \vec{\chi})^T \cdot (D - W) \cdot (\vec{1} + \vec{\chi})}{k (\vec{1}^T \cdot D \cdot \vec{1})} + \frac{(\vec{1} - \vec{\chi})^T \cdot (D - W) \cdot (\vec{1} - \vec{\chi})}{(1 - k) \vec{1}^T \cdot D \cdot \vec{1}}.$$

Κάνοντας πράξεις και λαμβάνοντας υπόψη ότι :

$$\vec{1}^T \cdot (D - W) \cdot \vec{\chi} = \vec{\chi}^T \cdot (D - W) \cdot \vec{1},$$

η παραπάνω ισότητα γράφεται ως εξής:

$$4Ncut(A, B) = \frac{\vec{\chi}^T \cdot (D - W) \cdot \vec{\chi} + \vec{1}^T \cdot (D - W) \cdot \vec{1}}{k(1 - k) \vec{1}^T \cdot D \cdot \vec{1}} + \frac{2(1 - 2k) \vec{1}^T \cdot (D - W) \cdot \vec{\chi}}{k(1 - k) \vec{1}^T \cdot D \cdot \vec{1}}.$$

Έστω τώρα:

$$\alpha(\vec{\chi}) = \vec{\chi}^T \cdot (D - W) \cdot \vec{\chi},$$

$$\beta(\vec{\chi}) = \vec{1}^T \cdot (D - W) \cdot \vec{\chi},$$

$$\gamma = \vec{1}^T \cdot (D - W) \cdot \vec{1},$$

$$M = \vec{1}^T \cdot D \cdot \vec{1},$$

τότε:

$$\begin{aligned} Ncut(A, B) &= \frac{\alpha(\vec{\chi}) + \gamma + 2(1 - 2k) \beta(\vec{\chi})}{k(1 - k) M} \\ &= \frac{\alpha(\vec{\chi}) + \gamma + 2(1 - 2k) \beta(\vec{\chi})}{k(1 - k) M} - \frac{2(\alpha(\vec{\chi}) + \gamma)}{M} + \frac{2\alpha(\vec{\chi})}{M} + \frac{2\gamma}{M}. \end{aligned}$$

Εφόσον ο τελευταίος όρος μηδενίζεται διότι  $\gamma=0$  , έχουμε:

$$\begin{aligned} Ncut(A, B) &= \frac{(\alpha(\vec{\chi}) + \gamma)(1 - 2k + 2k^2) + 2(1 - 2k) \beta(\vec{\chi})}{k(1 - k) M} + \frac{2\alpha(\vec{\chi})}{M} \\ &= \frac{\frac{1-2k+2k^2}{(1-k)^2} (\alpha(\vec{\chi}) + \gamma) + \frac{2(1-2k)}{(1-k)^2} \beta(\vec{\chi})}{\frac{k}{1-k} M} + \frac{2\alpha(\vec{\chi})}{M}. \end{aligned}$$

Θέτουμε

$$b = \frac{k}{1 - k}$$

και εφόσον  $\gamma=0$  παίρνουμε:

$$\begin{aligned} 4Ncut(A, B) &= \frac{(1 + b^2) + (\alpha(\vec{\chi}) + \gamma) + 2(1 - b^2) \beta(\chi)}{bM} + \frac{2b\alpha(\vec{\chi})}{bM} \\ &= \frac{(1 + b^2) + (\alpha(\vec{\chi}) + \gamma) + 2(1 - b^2) \beta(\chi)}{bM} + \frac{2b\alpha(\vec{\chi})}{bM} - \frac{2b\gamma}{bM} \\ &= \frac{(1 - b^2)(\chi^T \cdot (D - W) \cdot \vec{\chi}) + (\vec{1}^T \cdot (D - W) \cdot \vec{1})}{b (\vec{1}^T \cdot D \cdot \vec{1})} \\ &+ \frac{2(1 - b^2) (\vec{1}^T \cdot (D - W) \cdot \vec{\chi})}{b(\vec{1}^T \cdot D \cdot \vec{1})} \end{aligned}$$

$$\begin{aligned}
& + \frac{2b (\vec{\chi}^T \cdot (D - W) \cdot \vec{\chi})}{b (\vec{1}^T \cdot D \cdot \vec{1})} - \frac{2b (\vec{1}^T \cdot (D - W) \cdot \vec{1})}{b (\vec{1}^T \cdot D \cdot \vec{1})} \\
& = \frac{(\vec{1} + \vec{\chi}^T) \cdot (D - W) \cdot (\vec{1} + \vec{\chi})}{b (\vec{1}^T \cdot D \cdot \vec{1})} \\
& + \frac{b^2 (\vec{1} - \vec{\chi}^T) \cdot (D - W) \cdot (\vec{1} - \vec{\chi})}{b (\vec{1}^T \cdot D \cdot \vec{1})} \\
& + \frac{2b (\vec{1} - \vec{\chi}^T) \cdot (D - W) \cdot (\vec{1} + \vec{\chi})}{b (\vec{1}^T \cdot D \cdot \vec{1})} \\
& = \frac{[(\vec{1} + \vec{\chi}) - b (\vec{1} - \vec{\chi})]^T \cdot (D - W) \cdot [(\vec{1} + \vec{\chi}) - b (\vec{1} - \vec{\chi})]}{b (\vec{1}^T \cdot D \cdot \vec{1})}.
\end{aligned}$$

Θέτοντας

$$\vec{y} = (\vec{1} + \vec{\chi} - b (\vec{1} - \vec{\chi})) = \begin{cases} 2, \chi_i = 1 \\ -2b, \chi_i = -1 \end{cases},$$

έχουμε:

$$y^T \cdot D \cdot \vec{1} = y^T d = 2 \left( \sum_{x_i > 0} d_i - b \sum_{x_i < 0} d_i \right) = 2k \vec{1}^T \cdot D \cdot \vec{1} - \frac{2k}{1-k} (1-k) \vec{1}^T \cdot D \cdot \vec{1} = 0.$$

Έτσι λοιπόν:

$$\min_{\vec{\chi}: \chi_i = \pm 1} Ncut(\vec{\chi}) = \min_{\vec{y}} \frac{\vec{y}^T \cdot (D - W) \cdot \vec{y}}{\vec{y}^T \cdot D \cdot \vec{y}}, \quad (2.1)$$

έτσι ώστε  $y(i) \in \{1, \beta\}$  και επιπλέον

$$\vec{y}^T \cdot D \cdot \vec{1} = 0.$$

Η έκφραση αυτή είναι το πηλίκο Rayleigh-Ritz, βλέπε [2]. Αν θεωρήσουμε ότι οι τιμές του  $\vec{y}$  δεν είναι διακριτές, αλλά πραγματικές, τότε το πρόβλημα (2.1) επιλύεται απο το γενικευμένο πρόβλημα ιδιοτιμών:

$$\begin{aligned}
(D - W) \cdot y &= \lambda D \cdot y \Rightarrow D^{-\frac{1}{2}} \cdot (D - W) \cdot D^{-\frac{1}{2}} \cdot D^{\frac{1}{2}} \cdot y = \lambda D^{\frac{1}{2}} \cdot y \\
&\Rightarrow D^{-\frac{1}{2}} \cdot (D - W) \cdot D^{-\frac{1}{2}} \cdot z = \lambda z.
\end{aligned} \quad (2.2)$$

Τότε:

$$\vec{z}_0 = D^{\frac{1}{2}} \cdot \vec{1}$$

είναι ιδιοδιάνυσμα του (2.2) με ιδιοτιμή 0. Ο πίνακας  $D^{-\frac{1}{2}}(D - W)D^{-\frac{1}{2}}$  είναι Ερμητιανός και θετικά ημιορισμένος, άρα όλα τα ιδιοδιανύσματα είναι κάθετα μεταξύ τους. Έτσι το  $\vec{z}_1$  ιδιοδιάνυσμα που αντιστοιχεί στη 2η μικρότερη ιδιοτιμή είναι κάθετο στο  $\vec{1}$ . Πηγαίνοντας στην (2.1) έχουμε ότι το  $\vec{1}$  είναι ιδιοδιάνυσμα για  $\lambda = 0$  και

$$\vec{z}_1^T \cdot \vec{z}_0 = \vec{y}_1^T \cdot D \cdot \vec{1}.$$

Υπενθυμίζουμε ότι αν  $A$  πραγματικός συμμετρικός πίνακας, τότε υπό την προϋπόθεση ότι  $\vec{\chi}_j$  κάθετο στα ιδιοδιανύσματα  $\vec{\chi}_1, \dots, \vec{\chi}_{j-1}$  το πηλίκο:

$$\frac{\vec{\chi}^T \cdot A \cdot \vec{\chi}}{\vec{\chi}^T \cdot \vec{\chi}}$$

ελαχιστοποιείται απο το ιδιοδιάνυσμα  $\vec{\chi}_j$ , βλέπε [2]. Δηλαδή:

$$\vec{z}_1 = \arg \min_{\vec{z}^T \vec{z}_0} \frac{\vec{z}^T \cdot D^{-\frac{1}{2}} \cdot (D - W) D^{-\frac{1}{2}} \cdot \vec{z}}{\vec{z}^T \cdot \vec{z}}$$

και έτσι:

$$\vec{y}_1 = D^{-\frac{1}{2}} \cdot \vec{z}_1$$

είναι το σημείο ελαχίστου που ψάχνουμε. Παίρνουμε τα πρόσημα αυτού για να διακριτοποιήσουμε το πρόβλημα και να επιτύχουμε διαμέριση βέλτιστη. Μπορούμε να συνεχίσουμε παίρνοντας την τρίτη ιδιοτιμή και το τρίτο ιδιοδιάνυσμα που διαμερίζει βέλτιστα τις αρχικές υποδιαμερίσεις. Καλύτερα όμως να επιλύσουμε το πρόβλημα σε κάθε μία απο τις υποδιαμερίσεις απο την αρχή, διότι το σφάλμα συσσωρεύεται. Παραθέτουμε τον ψευδοκώδικα:

---

### Αλγόριθμος 3 Normalized Cut algorithm

---

- 1: Εστω σταθμισμένος γράφος  $G = (V, E)$  όπου το σύνολο  $V$  περιέχει τους κόμβους, ενώ το  $E$  περιέχει τις ακμές. Ετσι, δοθέντων των βαρών των ακμών μπορούμε να κατασκευάσουμε τους πίνακες  $W$  και  $D$ . Θέλουμε να διαμερίσουμε το γράφημα σε 2 υποδιαμερίσεις.
  - 2: Επιλύουμε το σύστημα  $D^{-\frac{1}{2}}(D - W)D^{-\frac{1}{2}}y = \lambda y$  για να προσδιορίσουμε το ιδιοδιάνυσμα που αντιστοιχεί στην δεύτερη μικρότερη ιδιοτιμή  $\vec{z}_1$ .
  - 3: Από την παραπάνω θεωρία γνωρίζουμε ότι η παράμετρος  $Ncut(A, B)$  λαμβάνει την ελάχιστη τιμή της στο ιδιοδιάνυσμα  $\vec{y}_1 = D^{-\frac{1}{2}}\vec{z}_1$ . Προσεγγίζουμε τη λύση, χρησιμοποιώντας μόνον τα θετικά και αρνητικά πρόσημα του ιδιοδιανύσματος αυτού.
  - 4: Εάν επιθυμούμε, αποφασίζουμε αν η τρέχουσα διαμέριση θα πρέπει να υποδιαιρεθεί περαιτέρω, ενδεχομένως προχωρώντας σε περαιτέρω διαμέριση με αναδρομικό τρόπο.
- 

Παραθέτουμε τον κώδικα σε python.

---

```

1 import numpy as np
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 from scipy.linalg import eigh
5 from numpy import sqrt
6
7 def normalized_cut_with_metrics(G):
```

```

8     W = nx.to_numpy_array(G, weight='weight')
9     degrees = W.sum(axis=1)
10    D = np.diag(degrees)
11    D_inv_sqrt = np.diag(1.0 / sqrt(degrees))
12    L = D-W
13    L_sym = np.eye(len(W)) - D_inv_sqrt @ W @ D_inv_sqrt
14    eigvals, eigvecs = eigh(L_sym)
15    z1 = eigvecs[:, 1]
16    y1 = D_inv_sqrt @ z1
17    partition = y1 >= 0
18    set_A = np.where(partition)[0]
19    set_B = np.where(~partition)[0]
20
21    # Μετρικές
22    assoc_A_V = np.sum(W[set_A, :])
23    assoc_B_V = np.sum(W[set_B, :])
24    cut_AB = np.sum(W[np.ix_(set_A, set_B)])
25    ncut = (cut_AB / assoc_A_V) + (cut_AB / assoc_B_V)
26
27    return set_A, set_B, z1, y1, assoc_A_V, assoc_B_V, cut_AB,
    ↪ ncut
28
29    # Δημιουργία γράφου
30    N = 21
31    M = 35
32    G = nx.gnm_random_graph(N, M, seed=30)
33
34    for u, v in G.edges():
35        G[u][v]['weight'] = round(np.random.uniform(1.0, 1.0), 1) #
    ↪ βάρη = 1.0
36
37    # Υπολογισμός διαμέρισης και μετρικών
38    A, B, z1, y1, assoc_A_V, assoc_B_V, cut_AB, ncut =
    ↪ normalized_cut_with_metrics(G)
39
40    # Τοποθέτηση κόμβων
41    pos = nx.spring_layout(G, seed=42)
42    edge_labels = nx.get_edge_attributes(G, 'weight')
43
44    # Απεικόνιση αρχικού και τελικού γράφου
45    plt.figure(figsize=(12, 5))
46
47    # Αρχικός γράφος
48    plt.subplot(1, 2, 1)
49    nx.draw(G, pos, with_labels=True, node_color='lightblue',
    ↪ edge_color='gray')

```

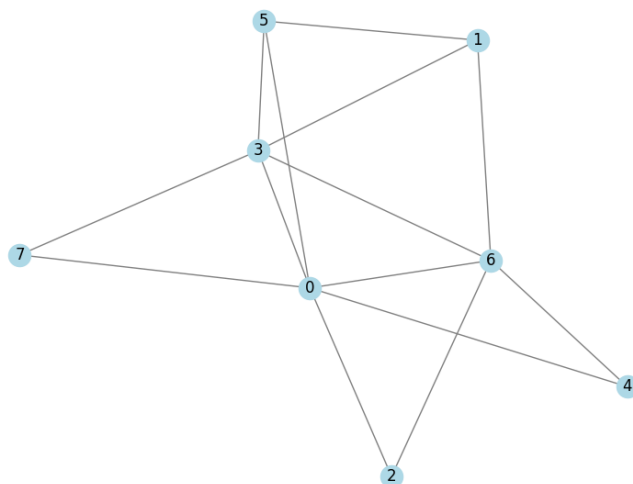
```

50 nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
51 plt.title('Αρχικός Γράφος')
52
53 # Τελικός γράφος μετά τον Normalized Cut
54 plt.subplot(1, 2, 2)
55 colors = ['orange' if node in A else 'green' for node in
    ↪ G.nodes()]
56 nx.draw(G, pos, with_labels=True, node_color=colors,
    ↪ edge_color='gray')
57 nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
58 plt.title('Μετά τον Normalized Cut')
59
60 plt.tight_layout()
61 plt.show()
62
63 # Εκτύπωση μετρικών
64 nodes_A = list(map(int, A))
65 nodes_B = list(map(int, B))
66 print("Nodes in A:", nodes_A)
67 print("Nodes in B:", nodes_B)
68 print("assoc(A, V):", assoc_A_V)
69 print("assoc(B, V):", assoc_B_V)
70 print("cut(A, B):", cut_AB)
71 print("Ncut:", ncut)

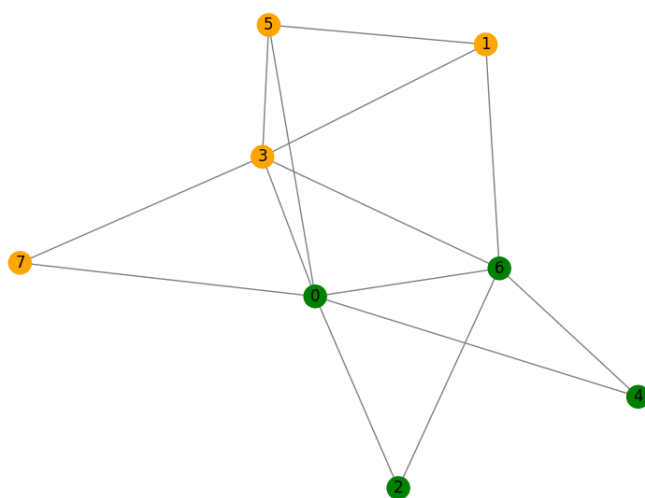
```

---

**Παράδειγμα 2.5.** Παραθέτουμε ένα παράδειγμα διαμέρισης γράφου με 8 κόμβους και 14 ακμές.

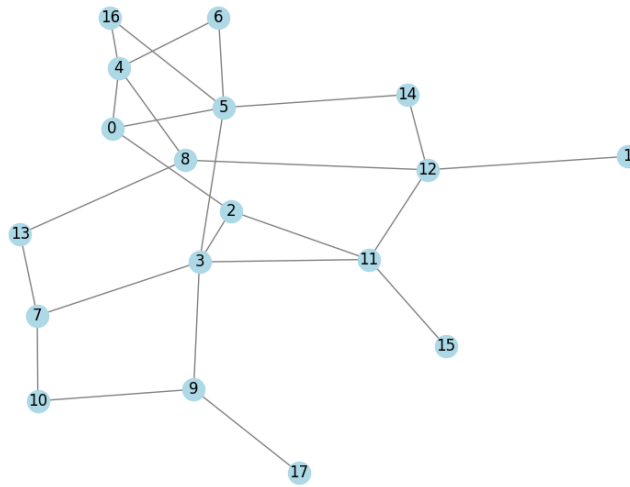


Σχήμα 2.10: Αρχικός γράφος.

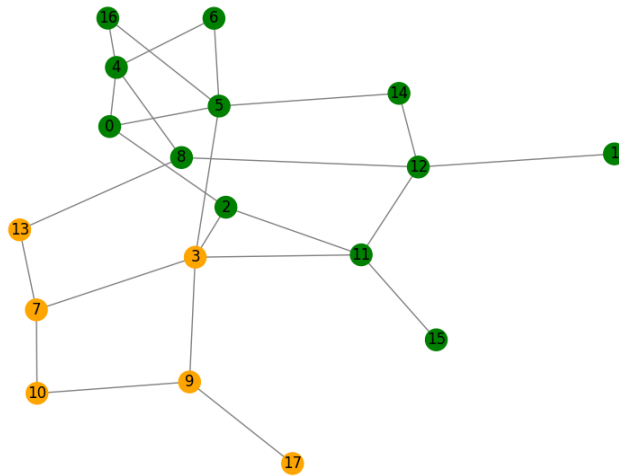


Σχήμα 2.11: Τελικός γράφος.

**Παράδειγμα 2.6.** Παραθέτουμε ένα παράδειγμα διαμέρισης γράφου με 18 κόμβους και 25 ακμές.



Σχήμα 2.12: Αρχικός γράφος.



Σχήμα 2.13: Τελικός γράφος.

## 3 Αλγόριθμος Kernighan–Lin

### 3.1 Εισαγωγή

Στο παρακάτω κεφάλαιο θα δούμε 2 αλγορίθμους διαμέρισης γράφου σε 2 υποσύνολα. Ο πρώτος είναι ο αλγόριθμος των Kernighan-Lin βλέπε [6] και ο δεύτερος είναι δική μας παραλλαγή.

Η διαμέριση γράφου είναι ένα θεμελιώδες πρόβλημα της συνδυαστικής βελτιστοποίησης και έχει στόχο την κατάτμηση ενός γράφου σε υποσύνολα κόμβων, σύμφωνα με ορισμένα κριτήρια. Συγκεκριμένα, ένας γράφος με βάρη στις ακμές, ζητείται να χωριστεί σε  $k$  υποσύνολα συγκεκριμένου μεγέθους, έτσι ώστε να ελαχιστοποιείται το συνολικό βάρος των ακμών που συνδέουν κόμβους διαφορετικών ομάδων. Εάν το γράφημα δεν είναι σταθμισμένο, τότε στόχος είναι να ελαχιστοποιηθεί ο αριθμός των ακμών που τέμνονται, δηλαδή που συνδέουν κόμβους διαφορετικών ομάδων. Επειδή το συνολικό άθροισμα του βάρους των ακμών είναι σταθερό, η ελαχιστοποίηση του εξωτερικού κόστους ταυτίζεται με τη μεγιστοποίηση του εσωτερικού κόστους. Πρόκειται για ένα πρόβλημα με εφαρμογή σε πεδία όπως η σχεδίαση κυκλωμάτων, η διαχείριση της μνήμης και γενικότερα η κατανομή αντικειμένων με πεπερασμένους πόρους. Παραθέτουμε κάποιες μεθόδους που έχουν προταθεί για τη λύση αυτού του προβλήματος

- Ακριβείς λύσεις: Για έναν γράφο με  $N$  κόμβους αν θέλαμε να τον διαμερίσουμε σε  $k$  ομάδες με  $p$  στοιχεία η καθεμιά, η εύρεση της βέλτιστης λύσης με το ελάχιστο κόστος θα απαιτούσε

$$\frac{1}{k!} \binom{n}{p} \binom{n-p}{p} \cdots \binom{2p}{p} \binom{p}{p}$$

συγκρίσεις. Μπορούμε να το γράψουμε και ως πρόβλημα Ακέραιου Γραμμικού Προγραμματισμού, χρησιμοποιώντας κατάλληλους περιορισμούς για να βρούμε την ακριβή λύση του προβλήματος. Ωστόσο, η υπολογιστική πολυπλοκότητα του προβλήματος για μεγάλο μέγεθος γράφου-ομάδων είναι απαγορευτική. Γι'αυτό τον λόγο στρεφόμαστε σε ευρετικούς αλγορίθμους.

- Ευρετικοί αλγόριθμοι: Αποσκοπούν στον υπολογισμό διαμέρισης όσο πιο κοντά γίνεται στη βέλτιστη(ή και την βέλτιστη) με μικρό υπολογιστικό κόστος. Δεν εξετάζουν όλες τις πιθανές λύσεις αλλά βασίζονται σε ευφυείς κανόνες ή τοπικές βελτιστοποιήσεις. Κάποιοι απο αυτούς είναι:
  1. Τυχαίες λύσεις: Παραγωγή τυχαίων διαμερίσεων και διατήρηση της καλύτερης. Έχουμε χαμηλή πολυπλοκότητα αλλά ειδικά για μεγάλους γράφους σπάνια παίρνουμε καλές διαμερίσεις.
  2. Μέγιστης ροής - ελάχιστης τομής: Προσεγγίζουμε τον γράφο σαν ένα δίκτυο όπου τα βάρη των ακμών αντιστοιχούν σε χωρητικότητες. Ο αλγόριθμος αυτός βρίσκει μια τομή με ελάχιστο κόστος, που είναι

ισοδύναμο με τη μέγιστη ροή από μία κορυφή σε μια άλλη. Δεν εξασφαλίζει ισορροπημένες διαμερίσεις αλλά παρέχει ένα κάτω όριο για το ελάχιστο κόστος. Δίνει πληροφορίες χρήσιμες για την χρήση του σε άλλες μεθόδους.

3. Συσταδοποίηση: Βασίζεται στον εντοπισμό ομάδων κόμβων που έχουν ισχυρή σύνδεση μεταξύ τους. Ωστόσο, δε διασφαλίζει την ισομερή υποσύνολα ούτε μπορεί να αντιμετωπίσει κόμβους που δεν ανήκουν καθαρά σε κάποια συστάδα.
4. λ-opting: Αυτή η μέθοδος βελτιώνει μια υπάρχουσα λύση μέσω ανταλλαγής κόμβων μεταξύ υποσυνόλων ώστε να μειωθεί το κόστος μεταξύ. Ο φυσικός αριθμός λ δείχνει τον αριθμό των ανταλλαγών σε κάθε βήμα. Η μέθοδος είναι υπολογιστικά ανεφάρμοστη για  $\lambda > 1$ . Παρόλα αυτά, αποτελεί το πρώτο βήμα στη λογική του αλγορίθμου που εφάρμοσαν οι *Kernighan – Lin* στο πρόβλημα της διαμερίσης.

Ξεκινάμε με μία αυθαίρετη διαμέριση ενός γράφου  $G = (V, E)$  με  $V = 2n$  κόμβους σε δυο ισάριθμα σύνολα  $A \cup B = G$ ,  $A \cap B = \emptyset$  και  $|A| = |B| = n$ . Σκοπός είναι να μειωθεί το συνολικό κόστος μέσω αμοιβαίων ανταλλαγών κόμβων μεταξύ των δύο συνόλων. Οι ανταλλαγές γίνονται με τέτοιο τρόπο ώστε σε κάθε βήμα να επιλέγεται το ζεύγος κόμβων που προσφέρει τη μέγιστη δυνατή μείωση του κόστους. Όταν καμία περαιτέρω βελτίωση δεν είναι δυνατή, η διαμέριση θεωρείται τοπικά βέλτιστη(ή είναι ολικά βέλτιστη). Ο αλγόριθμος επαναλαμβάνει τη διαδικασία βελτιστοποίησης ξεκινώντας τώρα από τα νέα σύνολα  $A^*$  και  $B^*$  που προέκυψαν από τις ανταλλαγές.

**Ορισμός 3.1.** Ορίζουμε έναν πίνακα κόστους  $C = c_{ij} \in \mathbb{R}^{2n \times 2n}$  που ταυτίζεται με τον πίνακα  $W$  της (1.2) όπου η διαφορά έγκειται στην ερμηνεία του στοιχείου  $c_{ij}$ , ως κόστος (αντί βάρους) μεταξύ κόμβων στο πλαίσιο της διαμερίσης. Έτσι λοιπόν  $W = C$ , όπου:

$$C_{ij} = \begin{cases} c_{ij}, & \text{αν } i \neq j \\ 0, & \text{αν } i = j \end{cases}.$$

**Ορισμός 3.2.** Εστω γράφος  $G$  με  $V = 2n$  κόμβους και  $A \cup B = G$ ,  $A \cap B = \emptyset$  και  $|A| = |B| = n$ . Για κάθε κόμβο  $i \in A$ , ορίζουμε ως εξωτερικό κόστος  $E_i$  να είναι το άθροισμα των ακμών από τον κόμβο  $i$  προς κόμβους του  $B$ :

$$E_i = \sum_{j \in B} c_{ij}.$$

Αντιοίστοχα ορίζουμε και το εσωτερικό κόστος  $I_i$  να είναι το άθροισμα των ακμών από τον κόμβο  $i$  προς κόμβους του ίδιου υποσυνόλου:

$$I_i = \sum_{j \in A} c_{ij}$$

και η διαφορά κόστους είναι :

$$D_i = E_i - I_i.$$

Το συνολικό κόστος υπολογίζεται ως εξής:

$$T = \sum_{\substack{i \in A \\ j \in B}} c_{ij}.$$

Για οποιοδήποτε ζεύγος κόμβων  $a \in A$ ,  $b \in B$ , το κέρδος από την ανταλλαγή τους είναι:

$$gain = D_a + D_b - 2c_{a,b}.$$

**Απόδειξη** Έστω  $\chi = \sum_{\substack{i \in A \setminus \{a\} \\ j \in B \setminus \{b\}}} c_{ij}$ . Τότε το συνολικό κόστος είναι:

$$T_1 = \chi + E_a + E_b - c_{ab}.$$

Μετά την ανταλλαγή των κόμβων  $a$  και  $b$  το καινούργιο κόστος είναι:

$$T_2 = \chi + I_a + I_b + c_{ab}.$$

Τελικά έχουμε:

$$gain = T_1 - T_2 = \chi - \chi + (E_a - I_a) + (E_b - I_b) - 2c_{ab} = D_a + D_b - 2c_{ab}.$$

□

Η διαδικασία έχει ως εξής:

1. Υπολογίζουμε τις τιμές  $D_i$  για όλους τους κόμβους.
2. Επιλέγουμε το ζεύγος  $(a_1, b_1)$  με το μέγιστο δυνατό κέρδος, π.χ.  $g_1$ .
3. Θέτουμε προσωρινά εκτός το  $(a_1, b_1)$  και ενημερώνουμε τις τιμές  $D_i$  για τα υπόλοιπα στοιχεία, σύμφωνα με:

$$D_x := D_x + 2c_{x,a_1} - 2c_{x,b_1}, \quad \text{για } x \in A \setminus \{a_1\}$$

$$D_y := D_y + 2c_{y,b_1} - 2c_{y,a_1}, \quad \text{για } y \in B \setminus \{b_1\}.$$

4. Επαναλαμβάνουμε τη διαδικασία για  $(a_2, b_2), \dots, (a_n, b_n)$ , αποθηκεύοντας τα αντίστοιχα  $g_2, \dots, g_n$ .
5. Υπολογίζουμε τα μερικά αθροίσματα  $G_k = \sum_{i=1}^k g_i$  για κάθε  $k = 1, \dots, n$  και βρίσκουμε το  $k$  που μεγιστοποιεί το συνολικό όφελος.
6. Αν  $G_k > 0$ , πραγματοποιούμε τις πρώτες  $k$  ανταλλαγές και ξανατρέχουμε την διαδικασία από τη νέα διαμέριση. Σε αντιθετη περίπτωση ο αλγόριθμος τερματίζει.

Η νέα διαμέριση μπορεί πλέον να επανεισαχθεί ως αρχική, και η διαδικασία να επαναληφθεί. Αν σε κάποια επανάληψη το συνολικό κέρδος  $G_k = 0$ , τότε η τρέχουσα διαμέριση θεωρείται τοπικά βέλτιστη.

Ο αλγόριθμος Kernighan–Lin έχει πολυπλοκότητα  $\mathcal{O}(n^2 \log n)$  ανά πλήρη επανάληψη, γεγονός που τον καθιστά κατάλληλο για χρήση σε μεσαίου μεγέθους γράφους. Αν και δεν εξαντλεί το σύνολο των πιθανών διαμερίσεων, έχει αποδειχθεί ιδιαίτερα αποτελεσματικός στην πρακτική εύρεση λύσεων χαμηλού κόστους.

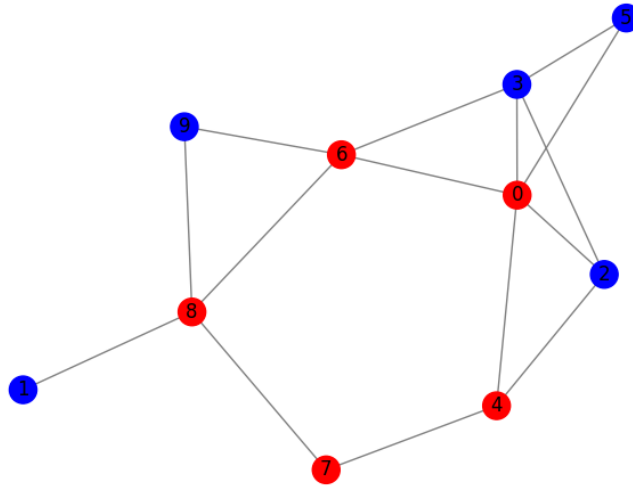
---

**Αλγόριθμος 4** Αλγόριθμος Kernighan–Lin

---

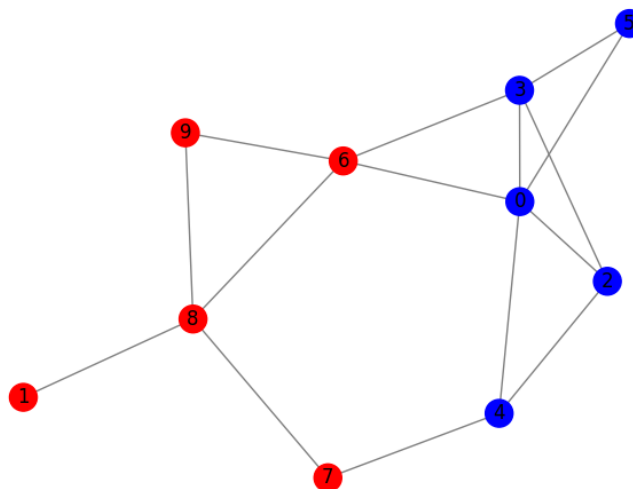
- 1: Παίρνουμε τυχαίο γράφημα  $G = (V, E)$  με  $2N$  κόμβους και  $M$  ακμές.
  - 2: Ορίζουμε τρία κενά σύνολα  $G, L, M$ .
  - 3: Ορίζουμε δύο ισάριθμα σύνολα  $A = \{1, \dots, N\}$  και  $B = \{N + 1, \dots, 2N\}$ .
  - 4: **for**  $i=1$  to  $N$  **do**
  - 5: Για κάθε  $a \in A, b \in B$  υπολογίζουμε το μέγιστο  $g$  της ποσότητας  $gain(a, b) = D(a) + D(b) - 2c(a, b)$ , όπου  $D(v) =$  συνολικό κόστος ακμών από  $v$  προς απέναντι υποσύνολο - συνολικό κόστος ακμών από  $v$  προς ίδιο υποσύνολο.
  - 6: Θέτουμε  $G \leftarrow G \cup \{g\}$ ,  $L \leftarrow L \cup \{a\}$ ,  $M \leftarrow M \cup \{b\}$  και  $A \leftarrow A \setminus L$  και  $B \leftarrow B \setminus M$ .
  - 7: Ενημερώνουμε τις τιμές  $D(v)$  για τα σύνολα  $A \setminus L$  και  $B \setminus M$ .
  - 8: **end for**
  - 9: Υπολογίζουμε τον φυσικό  $k$  ώστε η ποσότητα  $G(k) = g_1 + \dots + g_k$  να μεγιστοποιηθεί.
  - 10: **if**  $G(k) > 0$  **then**,
  - 11: Εφαρμόζουμε  $k$  ανταλλαγές  $a_1, \dots, a_k$  με τα αντίστοιχα  $b_1, \dots, b_k$  (και συνεπώς ορίζουμε νέα διαμέριση  $A$  και  $B$ ).
  - 12: Πηγαίνουμε στο Βήμα 4.
  - 13: **else**
  - 14: Τερματίζουμε τον αλγόριθμο και επιστρέφεται η διαμέριση μετά τις  $k$  ανταλλαγές.
  - 15: **end if**
-

**Παράδειγμα 3.1.** Παραθέτουμε ένα παράδειγμα διαμέρισης γράφου με 10 κόμβους και 15 ακμές. Ορίζουμε τυχαία αρχικά τα δυο σύνολα  $A = \{0, 4, 6, 7, 8\}$  και  $B = \{1, 2, 3, 5, 9\}$ . (Κόκκινο χρώμα για στοιχεία του συνόλου  $C$  και μπλε για του συνόλου  $B$ )



Σχήμα 3.1: Αρχικός γράφος.

Ο γράφος μετά την ανταλλαγή των στοιχείων:



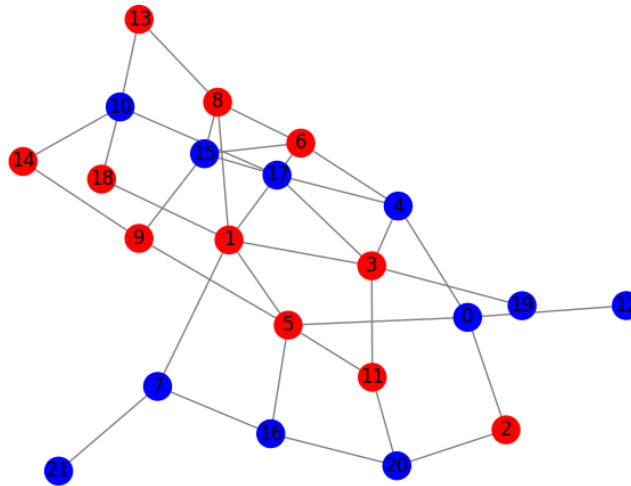
Σχήμα 3.2: Τελικός γράφος.

Τα νέα σύνολα που προκύπτουν είναι:  $A = \{1, 6, 7, 8, 9\}$  και  $B = \{0, 2, 3, 4, 5\}$ .

| Επανάληψη | Κόστος Τομής | Συνολικό Κέρδος |
|-----------|--------------|-----------------|
| 1         | 10.0         | 7.0             |
| 2         | 4.0          | 0.0             |

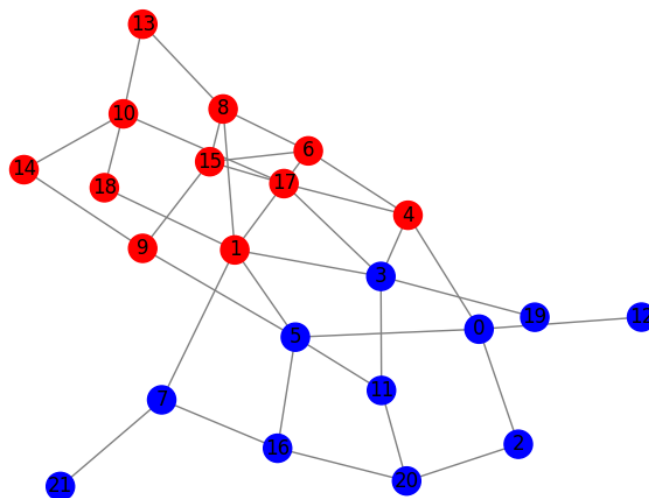
Πίνακας 1: Ανάλυση επαναλήψεων του αλγορίθμου

**Παράδειγμα 3.2.** Παραθέτουμε ένα παράδειγμα διαμέρισης γράφου με 22 κόμβους και 35 ακμές. Ορίζουμε τυχαία αρχικά τα δυο σύνολα  $A = \{1, 2, 3, 5, 6, 8, 9, 11, 13, 14, 18\}$  και  $B = \{0, 4, 7, 10, 12, 15, 16, 17, 19, 20, 21\}$ . (Κόκκινο χρώμα για στοιχεία του συνόλου  $A$  και μπλε για του συνόλου  $B$ .)



Σχήμα 3.3: Αρχικός γράφος.

Ο γράφος μετά την ανταλλάγή των στοιχείων:



Σχήμα 3.4: Τελικός γράφος.

Τα νέα σύνολα που προκύπτουν είναι:  $A = \{1, 4, 6, 8, 9, 10, 13, 14, 15, 17, 18\}$  και  $B = \{0, 2, 3, 5, 7, 11, 12, 16, 19, 20, 21\}$ .

| Επανάληψη | Κόστος Τομής | Συνολικό Κέρδος |
|-----------|--------------|-----------------|
| 1         | 16.0         | 7.0             |
| 2         | 9.0          | 1.0             |
| 3         | 8.0          | 1.0             |
| 4         | 7.0          | 0.0             |

Πίνακας 2: Ανάλυση επαναλήψεων του αλγορίθμου

### 3.2 Μια παραλλαγή του αλγορίθμου

Παρακάτω θα παραθέσουμε μια παραλλαγή του δικού μας κώδικα για το πρόβλημα της διαμέρισης. Παίρνουμε αρχικά έναν τυχαίο γράφο  $G = (V, E)$  με  $V = n$  κόμβους και 2 υποσύνολα  $C, B$  τέτοια ώστε,  $C \cup B = G, C \cap B = \emptyset$  και  $|C| = |B| = \frac{n}{2}$ . Στόχος μας είναι σε κάθε επανάληψη να μειώσουμε τη συνολική συνδεσιμότητα μεταξύ των δυο συνόλων, επιλέγοντας και ανταλλάσσοντας το ζεύγος κόμβων που οδηγεί στο μέγιστο κέρδος, δηλαδή τη μέγιστη μείωση του συνολικού κόστους της διαμέρισης. Ο αλγόριθμος σταματάει όταν καμία ανταλλαγή δεν αποδίδει περαιτέρω βελτίωση της διαμέρισης, δηλαδή μείωσης του συνολικού κόστους.

Έστω γράφος  $G$  με  $V = n$  κόμβους και  $C \cup B = G, C \cap B = \emptyset$  και  $|C| = |B| = \frac{n}{2}$ . Έχουμε:  $\alpha$  το πλήθος των στοιχείων του  $C$  και  $\beta$  το πλήθος των στοιχείων του  $B$ . Υπολογίζουμε τον πίνακα γειτνίασης  $A_{n \times n}$ .

**Ορισμός 3.3.** Για κάθε κόμβο  $c_i \in C$  και  $b_j \in B$ , ορίζουμε  $d_1(i)$  το πηλίκο:

$$d_1(i) = \frac{\sum_{j=1}^{\beta} A(c_i, b_j) - \sum_{\lambda=1}^{\alpha} A(c_i, c_{\lambda})}{0.001 + \sum_{\mu=1}^N A(c_i, \mu)}, i = 1, \dots, \alpha.$$

όπου το άθροισμα  $\sum_{j=1}^{\beta} A(c_i, b_j)$  εκφράζει τη συνδεσιμότητα του κόμβου  $c_i$  με όλα τα στοιχεία του  $B$ , το άθροισμα  $\sum_{\lambda=1}^{\alpha} A(c_i, c_{\lambda})$  εκφράζει τη συνδεσιμότητα του κόμβου  $c_i$  με όλα τα στοιχεία του  $C$  και το άθροισμα  $\sum_{j=1}^{\beta} A(c_i, b_j)$  εκφράζει τη συνδεσιμότητα του κόμβου  $c_i$  με όλα τα στοιχεία του  $G$ . Αντίστοιχα ορίζουμε:

$$d_2(j) = \frac{\sum_{i=1}^{\alpha} A(b_j, c_i) - \sum_{\lambda=1}^{\beta} A(b_j, b_{\lambda})}{0.001 + \sum_{\mu=1}^N A(b_j, \mu)}, j = 1, \dots, \beta.$$

Επίσης υπολογίζουμε:

$$dd(i, j) = d_1(i) + d_2(j) - \frac{2A(c_i, b_j)}{0.001 + \sum_{\mu=1}^N A(c_i, \mu)} - \frac{2A(b_j, c_i)}{0.001 + \sum_{\mu=1}^N A(b_j, \mu)},$$

$$i = 1, \dots, \alpha \text{ και } j = 1, \dots, \beta.$$

Τέλος υπολογίζουμε το συνολικό κόστος της διαμέρισης:

$$e = \sum_{i=1}^{\alpha} \sum_{j=1}^{\beta} A(c_i, b_j).$$

Η παραλλαγή του αλγορίθμου Kernighan-Lin λειτουργεί ως εξής:

1. Υπολογίζουμε τις τιμές  $d_1(i), d_2(j)$  και  $dd(i, j)$  για όλους τους κόμβους.
2. Υπολογίζουμε τις τιμές  $\rho = \max_{i,j} dd(i, j), e = \sum_{i=1}^{\alpha} \sum_{j=1}^{\beta} A(c_i, b_j)$ . και  $e' \leftarrow e$ .
3. Αν  $e' = 0$  ή  $\max_{i,j} dd(i, j) \leq 0$  σταματάμε. Αλλιώς:
4. Βρίσκουμε τις θέσεις  $(i_o, j_o)$  στις οποίες  $d(i_o, j_o) = \rho$  και αν υπάρχουν πολλές επιλέγουμε την πρώτη.
5. Αντιμεταθέτουμε το  $c_{i_o} \in C$  με το  $b_{j_o} \in B$
6. Η επαναλήψεις συνεχίζονται μέχρι να μην μπορεί να μειωθεί το συνολικό κόστος  $e - e' > 0$

---

#### Αλγόριθμος 5 Παραλλαγή Kernighan-Lin

---

- 1: Πάρε τυχαίο γράφημα  $N$  κόμβων και  $M$  ακμών.
  - 2: Όρισε δύο σύνολα  $C = \{1, \dots, \lfloor \frac{N}{2} \rfloor\}$  και  $B = \{\lfloor \frac{N}{2} \rfloor + 1, \dots, N\}$ .
  - 3: Υπολόγισε:  $a =$  πλήθος στοιχείων του  $C$ .
  - 4: Υπολόγισε:  $b =$  πλήθος στοιχείων του  $B$ .
  - 5: Υπολόγισε πίνακα γειτνίασης  $A_{N \times N}$ .
  - 6: **while**  $e - e' > 0$  **do**
  - 7: **for all**  $c_i \in C$  και  $b_j \in B$  **do**
  - 8:  $d_1(i) \leftarrow \frac{\sum_{j=1}^b A(c_i, b_j) - \sum_{\lambda=1}^a A(c_i, c_\lambda)}{0.001 + \sum_{\mu=1}^N A(c_i, \mu)}$
  - 9:  $d_2(j) \leftarrow \frac{\sum_{i=1}^a A(b_j, c_i) - \sum_{\lambda=1}^b A(b_j, b_\lambda)}{0.001 + \sum_{\mu=1}^N A(b_j, \mu)}$
  - 10:  $dd(i, j) \leftarrow d_1(i) + d_2(j) - \frac{2A(c_i, b_j)}{0.001 + \sum_{\mu=1}^N A(c_i, \mu)} - \frac{2A(b_j, c_i)}{0.001 + \sum_{\mu=1}^N A(b_j, \mu)}$
  - 11: **end for**
  - 12:  $\rho \leftarrow \max_{i,j} dd(i, j)$
  - 13:  $e \leftarrow \sum_{i=1}^a \sum_{j=1}^b A(c_i, b_j), \quad e' \leftarrow e$
  - 14: **if**  $\max_{i,j} dd(i, j) \leq 0 \vee e = 0$  **then**
  - 15: **Έξοδος:** Βέλτιστη ελαχιστοποίηση βαρών του γράφου.
  - 16: **else**
  - 17: Βρες  $(i_o, j_o)$  τέτοιο ώστε  $dd(i_o, j_o) = \rho$
  - 18: Αντιμετάθεσε το  $c_{i_o} \in C$  με το  $b_{j_o} \in B$  ώστε να δημιουργηθούν νέα σύνολα.
  - 19:  $C \leftarrow \{c_1, \dots, c_{i_o-1}, b_{j_o}, c_{i_o+1}, \dots, c_a\}$
  - 20:  $B \leftarrow \{b_1, \dots, b_{j_o-1}, c_{i_o}, b_{j_o+1}, \dots, b_b\}$
  - 21: **end if**
  - 22: **end while**
-

Παραθέτουμε και τον κώδικα σε python.

---

```
1 import numpy as np
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 import random
5 import pandas as pd
6
7 # --- Αρχικές παράμετροι ---
8 N = 10 # Αριθμός κόμβων
9 M = 15 # Αριθμός ακμών
10 G = nx.gnm_random_graph(n=N, m=M)
11
12 # Αντιστοίχιση τυχαίων βαρών στις ακμές
13 for u, v in G.edges():
14     G[u][v]['weight'] = np.round(np.random.uniform(1, 1), 2)
15
16 # Αρχικά τυχαία σύνολα C και B
17 nodes = list(G.nodes())
18 random.shuffle(nodes)
19 half = len(nodes) // 2
20 C = set(nodes[:half])
21 B = set(nodes[half:])
22
23 # Πίνακας γειτνίασης
24 A = nx.to_numpy_array(G, weight='weight')
25
26 def compute_cut_cost(G, C, B):
27     return sum(G[u][v]['weight'] for u in C for v in B if
28                 ↪ G.has_edge(u, v))
29
30 # Ιστορικό επαναλήψεων
31 history = []
32
33 # Αρχική τιμή κόστους
34 e = compute_cut_cost(G, C, B)
35 e_prev = float('inf')
36
37 # Διάταξη για σχεδίαση
38 pos = nx.spring_layout(G, seed=42)
39
40 # Εμφάνιση αρχικού γράφου
41 color_map = ['red' if node in C else 'blue' for node in
42              ↪ G.nodes()]
43 nx.draw(G, pos, with_labels=True, node_color=color_map,
44         ↪ edge_color='gray')
```

```

42 plt.title("Αρχικός Γράφος")
43 plt.show()
44
45 # --- Παράλλαξη Αλγορίθμου Kernighan{Lin} ---
46 iteration = 0
47 while e_prev - e > 0:
48     iteration += 1
49     e_prev = e
50     max_gain = float('-inf')
51     best_pair = None
52
53     for ci in list(C):
54         for bj in list(B):
55             deg_ci = np.sum(A[ci])
56             deg_bj = np.sum(A[bj])
57             d1 = (sum(A[ci][list(B)]) - sum(A[ci][list(C)])) /
58                 ↳ (0.001 + deg_ci)
59             d2 = (sum(A[bj][list(C)]) - sum(A[bj][list(B)])) /
60                 ↳ (0.001 + deg_bj)
61             dd = d1 + d2 - (2 * A[ci][bj]) / (0.001 + deg_ci) -
62                 ↳ (2 * A[bj][ci]) / (0.001 + deg_bj)
63
64             if dd > max_gain:
65                 max_gain = dd
66                 best_pair = (ci, bj)
67
68 # Καταγραφή στο ιστορικό
69 history.append({
70     "iteration": iteration,
71     "cut_cost": round(e, 2),
72     "max_gain": round(max_gain, 2),
73 })
74
75 if max_gain <= 0 or e == 0:
76     break
77
78 ci, bj = best_pair
79 C.remove(ci)
80 C.add(bj)
81 B.remove(bj)
82 B.add(ci)
83 e = compute_cut_cost(G, C, B)
84
85 # --- Εμφάνιση τελικού γράφου ---

```

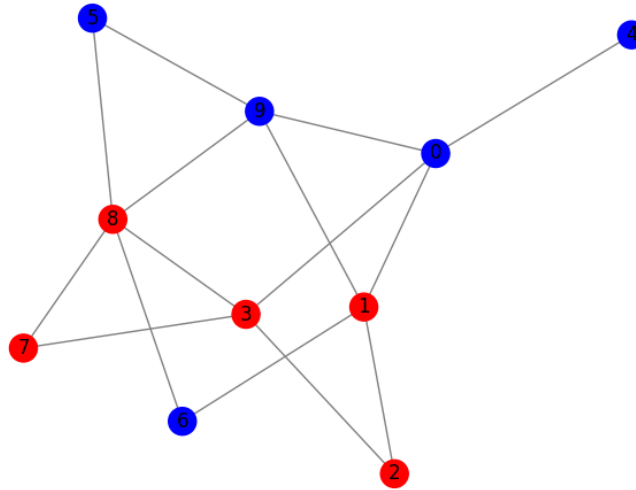
```

84 final_color_map = ['red' if node in C else 'blue' for node in
    ↪ G.nodes()]
85 nx.draw(G, pos, with_labels=True, node_color=final_color_map,
    ↪ edge_color='gray')
86 plt.title("Τελικός Γράφος")
87 plt.show()
88
89
90 # --- Εκτύπωση Πίνακα Ιστορικού ---
91 df = pd.DataFrame(history)
92 print("\nΑνάλυση επαναλήψεων:")
93 print(df.to_string(index=False))
94

```

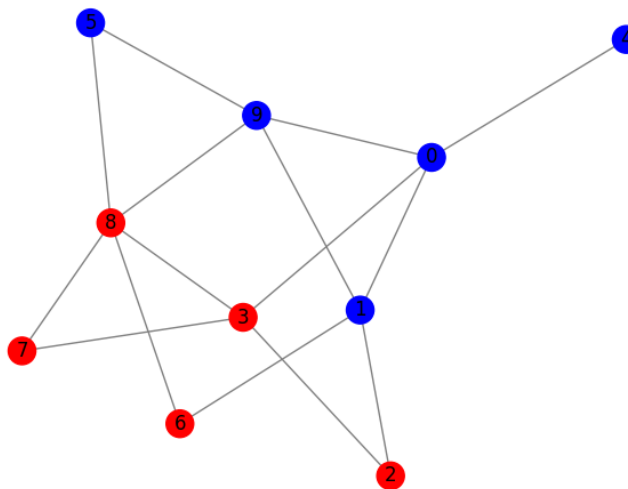
---

**Παράδειγμα 3.3.** Παραθέτουμε ένα παράδειγμα διαμέρισης γράφου με 10 κόμβους και 15 ακμές. Ορίζουμε τυχαία αρχικά τα δυο σύνολα  $C = \{1, 2, 3, 6, 7\}$  και  $B = \{0, 4, 5, 8, 9\}$ . (Κόκκινο χρώμα για στοιχεία του συνόλου  $C$  και μπλε για του συνόλου  $B$ )



Σχήμα 3.5: Αρχικός γράφος.

Ο γράφος μετά την ανταλλάγή των στοιχείων:



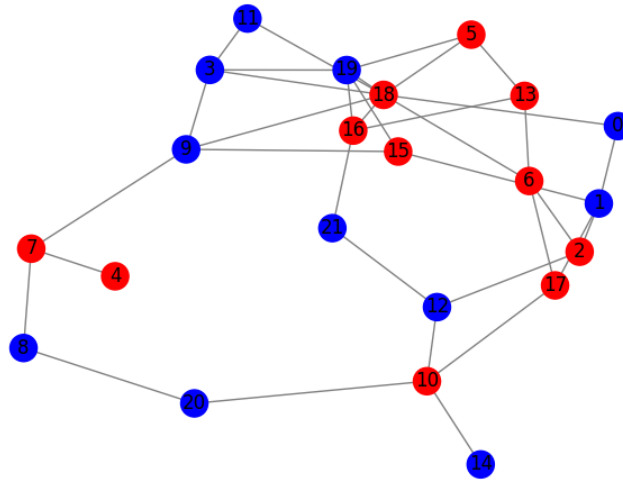
Σχήμα 3.6: Τελικός γράφος.

Τα νέα σύνολα που προκύπτουν είναι:  $C = \{2, 4, 6, 7, 8\}$  και  $B = \{0, 1, 3, 5, 9\}$ .

| Επανάληψη | Κόστος Τομής | Μέγιστο Κέρδος |
|-----------|--------------|----------------|
| 1         | 7.0          | 1.0            |
| 2         | 5.0          | 0.0            |

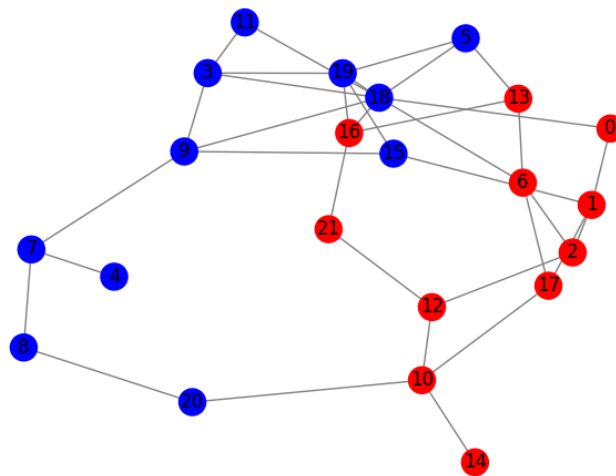
Πίνακας 3: Ανάλυση επαναλήψεων του αλγορίθμου

**Παράδειγμα 3.4.** Παραθέτουμε τώρα ένα παράδειγμα διαμέρισης γράφου με 22 κόμβους και 35 ακμές. Ορίζουμε αρχικά τα δυο σύνολα  $C = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$  και  $B = \{11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21\}$ . (Κόκκινο χρώμα για στοιχεία του συνόλου  $A$  και μπλε για του συνόλου  $B$ )



Σχήμα 3.7: Αρχικός γράφος.

Ο γράφος μετά την ανταλλαγή των στοιχείων:



Σχήμα 3.8: Τελικός γράφος.

Τα νέα σύνολα που προκύπτουν είναι:  $C = \{0, 1, 2, 4, 5, 8, 9, 10, 12, 14, 17\}$  και  $B = \{3, 6, 7, 11, 13, 15, 16, 18, 19, 20, 21\}$ . Το αρχικό κόστος είναι :  $e = 18$  ενώ το τελικό  $e = 6$ .

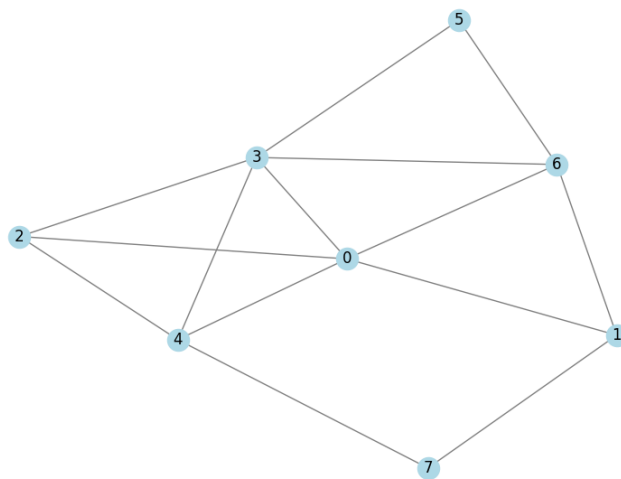
| Επανάληψη | Κόστος Τομής | Μέγιστο Κέρδος |
|-----------|--------------|----------------|
| 1         | 19.0         | 2.00           |
| 2         | 15.0         | 0.67           |
| 3         | 13.0         | 2.00           |
| 4         | 10.0         | 0.25           |
| 5         | 8.0          | 0.33           |
| 6         | 7.0          | 0.00           |

Πίνακας 4: Ανάλυση επαναλήψεων του αλγορίθμου

### 3.3 Σύγκριση μεταξύ των αλγορίθμων

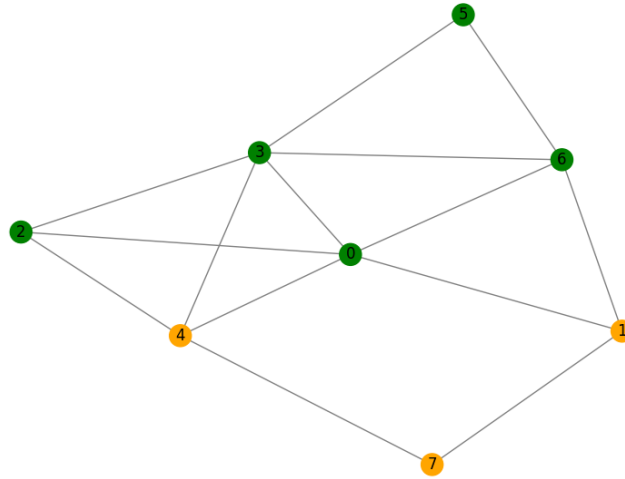
Παρακάτω θα κάνουμε μια σύγκριση των αλγορίθμων N-cut, Kernighan-Lin και της δικής μας τροποποίησης χρησιμοποιώντας ως σημείο αναφοράς τον ίδιο γράφο. Επίσης, για τους αλγορίθμους του κεφαλαίου 3, δεν θα πάρουμε τυχαία τα 2 σύνολα αλλά με αριθμητική σειρά για καλύτερη σύγκριση (π.χ για  $N = 6$ ,  $A = \{1, 2, 3\}$  και  $B = \{4, 5, 6\}$ ).

**Παράδειγμα 3.5.** Αυτός θα είναι ο γράφος αναφοράς:  $N = 8, M = 14$  και  $seed = 42$ .



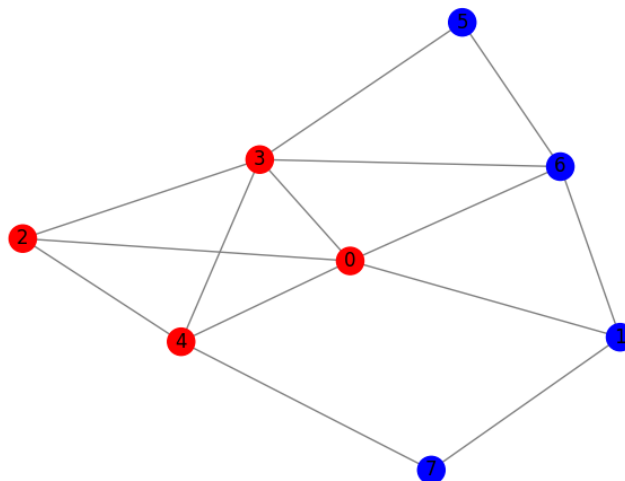
Σχήμα 3.9: Αρχικός γράφος.

*Χρήση του αλγορίθμου  $N - Cut$ .*



Σχήμα 3.10: Τελικός γράφος με  $N - Cut$ .

Χρήση του αλγορίθμου  $Kernighan - Lin$ .

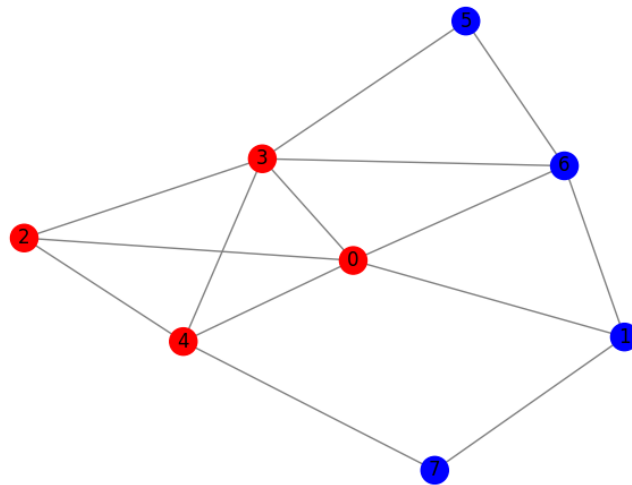


Σχήμα 3.11: Τελικός γράφος με  $Kernighan - Lin$ .

Πίνακας 5: Ανάλυση Επαναλήψεων

| Επανάληψη | Βάρος Τομής | Συνολικό Όφελος |
|-----------|-------------|-----------------|
| 1         | 8.0         | 3.0             |
| 2         | 5.0         | -1.0            |

Χρήση της δικής μας παραλλαγής.

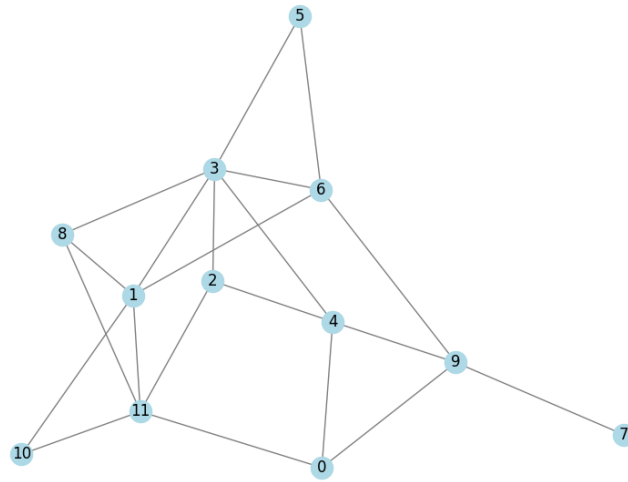


Σχήμα 3.12: Τελικός γράφος με δική μας παραλλαγή.

Πίνακας 6: Ανάλυση Επαναλήψεων

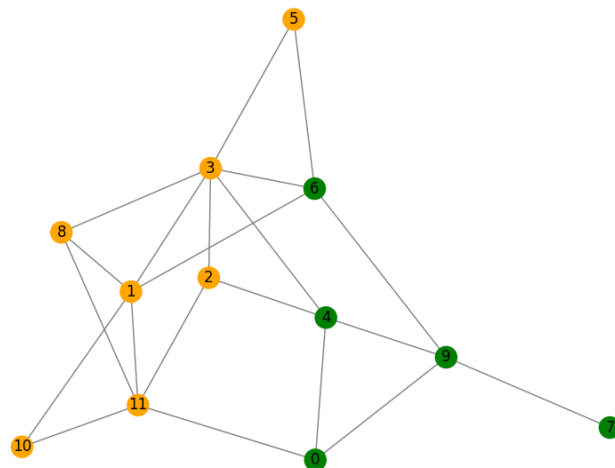
| Επανάληψη | Βάρος Τομής | Μέγιστο Κέρδος |
|-----------|-------------|----------------|
| 1         | 8.0         | 1.0            |
| 2         | 5.0         | -0.2           |

**Παράδειγμα 3.6.** Αυτός θα είναι ο γράφος αναφοράς:  $N = 12, M = 21$  και  $seed = 42$ .



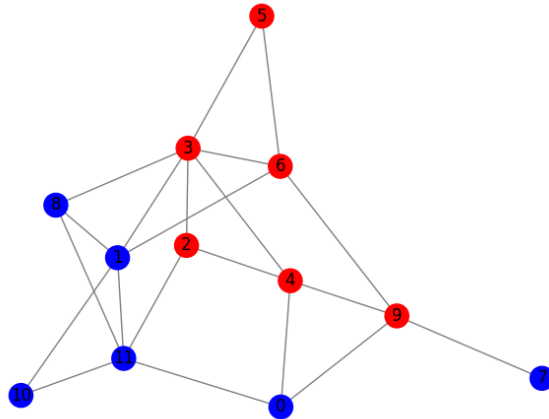
Σχήμα 3.13: Αρχικός γράφος.

Χρήση του αλγορίθμου  $N - Cut$ .



Σχήμα 3.14: Τελικός γράφος με  $N - Cut$ .

Χρήση του αλγορίθμου *Kernighan – Lin*.

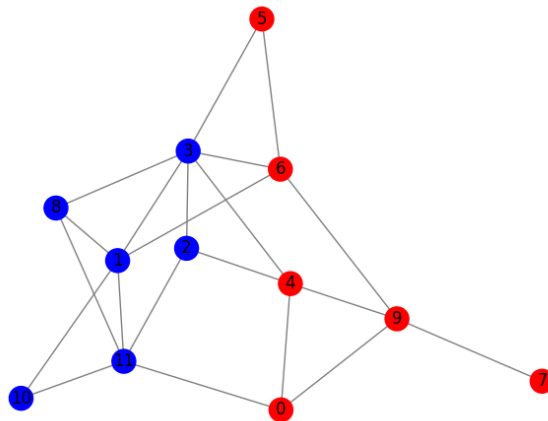


Σχήμα 3.15: Τελικός γράφος με *Kernighan – Lin*.

| Επανάληψη | Κόστος Τομής | Συνολικό Κέρδος |
|-----------|--------------|-----------------|
| 1         | 11.0         | 5.0             |
| 2         | 7.0          | 0.0             |

Πίνακας 7: Ανάλυση Επαναλήψεων

Χρήση της δικής μας παραλλαγής.

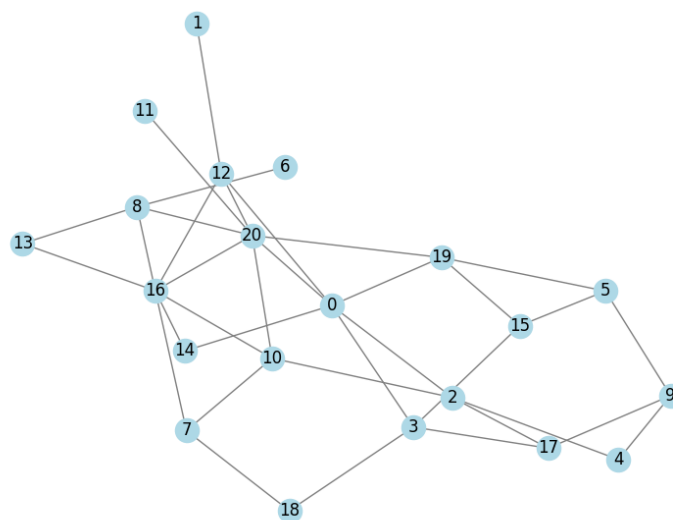


Σχήμα 3.16: Τελικός γράφος με δική μας παραλλαγή.

| Επανάληψη | Κόστος Τομής | Μέγιστο Κέρδος |
|-----------|--------------|----------------|
| 1         | 10.0         | 1.33           |
| 2         | 8.0          | 0.33           |
| 3         | 7.0          | 0.33           |
| 4         | 6.0          | -0.33          |

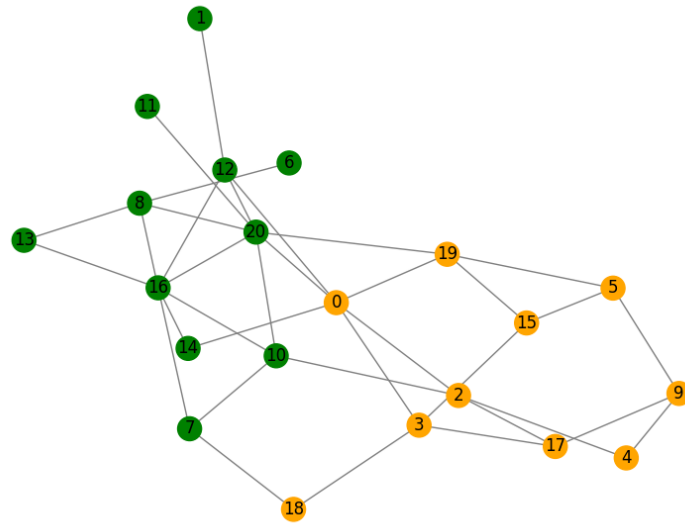
Πίνακας 8: Ανάλυση Επαναλήψεων

**Παράδειγμα 3.7.** Αυτός θα είναι ο γράφος αναφοράς:  $N = 21$ ,  $M = 35$  και  $seed = 30$ .



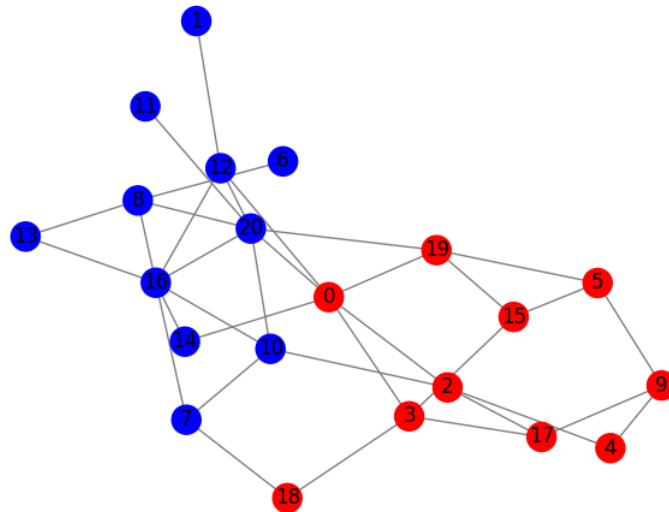
Σχήμα 3.17: Αρχικός γράφος.

Χρήση του αλγορίθμου  $N - Cut$ .



Σχήμα 3.18: Τελικός γράφος με  $N - Cut$ .

Χρήση του αλγορίθμου  $Kernighan - Lin$ .

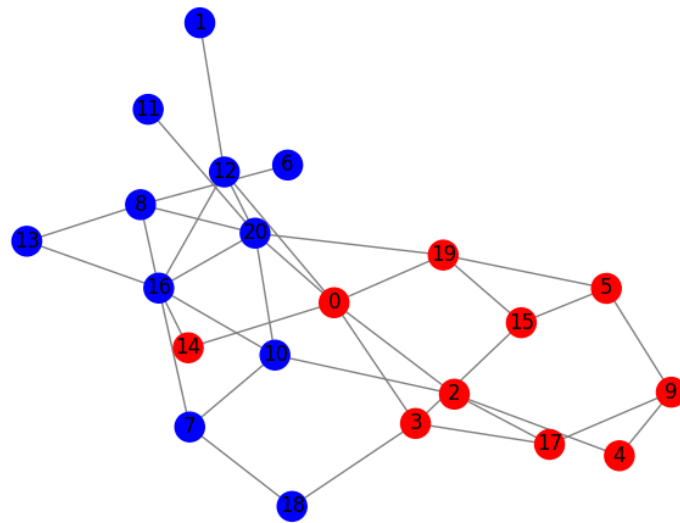


Σχήμα 3.19: Τελικός γράφος με  $Kernighan - Lin$ .

| Επανάληψη | Κόστος Τομής | Συνολικό Κέρδος |
|-----------|--------------|-----------------|
| 1         | 19.0         | 13.0            |
| 2         | 7.0          | 1.0             |
| 3         | 6.0          | 0.0             |

Πίνακας 9: Ανάλυση Επαναλήψεων

Χρήση της δικής μας παραλλαγής.



Σχήμα 3.20: Τελικός γράφος με δική μας παραλλαγή.

| Επανάληψη | Κόστος Τομής | Μέγιστο Κέρδος |
|-----------|--------------|----------------|
| 1         | 19.0         | 2.00           |
| 2         | 13.0         | 1.33           |
| 3         | 11.0         | 1.00           |
| 4         | 7.0          | 1.00           |
| 5         | 6.0          | 0.00           |

Πίνακας 10: Ανάλυση Επαναλήψεων

## Αναφορές

- [1] Σ.Δ.Νικολόπουλος, Α.Γεωργιάδης, Α.Παληός. Αλγοριθμική Θεωρία Γραφημάτων. Ελληνικά Ακαδημαϊκά Ηλεκτρονικά Συγγράματα και Βοηθήματα. 2015
- [2] R. Horn and C. Johnson. Matrix Analysis. Cambridge University Press, 1985.
- [3] L. Stankovic, M. Dakovic, and Ervin Sejdic. Introduction to Graph Signal Processing. Published in: L. Stankovic and E. Sejdic (eds), Vertex-Frequency Analysis of Graph Signals, Springer-Nature, 2019.
- [4] U. v. Luxburg. A Tutorial on Spectral Clustering. Statistics and Computing, 17 (4), 2007.
- [5] J. Shi, J. Malik. Normalized Cuts and Image Segmentation. IEEE Trans. on Pattern Analysis and Machine Intelligence, 22 (8), 2000, 888-905
- [6] B. W. Kernighan, S.Lin. An Efficient Heuristic Procedure for Partitioning Graphs. Bell System Technical Journal, 49, 1970, 291-307.